Uniform Grid: Neighbor Search Acceleration Structures for SPH Fluid Simulation

Submitted to the

Albert-Ludwigs-Universität Freiburg

by Nahhas Amal

Erstgutachter: Dr.-Ing. Matthias Teschner



universität freiburg

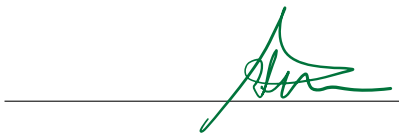
Declaration

I hereby declare, that I am the sole author and composer of my thesis and that no other sources or learning aids, other than those listed, have been used. Furthermore, I declare that I have acknowledged the work of others by providing detailed references of said work.

I also hereby declare that my thesis has not been prepared for another examination or assignment, either in its entirety or excerpts thereof.

Freiburg i. Br. 25.9.26

Place, date



Signature

Contents

1 Introduction **5**
1.1 Motivation . . . 5
1.2 Goals of the Thesis . . . 5
1.3 Overview . . . 5

2 SPH Fluid Simulation **6**
2.1 Governing Equations . . . 6
2.2 Smoothed-Particle Hydrodynamics . . . 7
2.3 SPH Quantities . . . 8
2.4 SPH Fluid Solver . . . 11

3 Neighbor Search **15**
3.1 Uniform Grid . . . 16
3.2 Index Sort . . . 17
3.2.1 Indexing . . . 19
3.2.2 Sorting . . . 20
3.3 Hashing . . . 22
3.3.1 Spatial Hashing . . . 22
3.3.2 Compact Hashing . . . 24

4 Neighbor Representation **24**
4.1 Compression . . . 24

5 Results **26**
5.1 Neighbor Search Analysis . . . 27
5.1.1 Static Scenes . . . 27
5.1.2 Dynamic Scenes . . . 34
5.2 Neighbor Representation Analysis . . . 39
5.2.1 Compression . . . 39

6 Conclusions **40**

A Boundary Handling **42**
A.1 Mirroring . . . 42

A.2 Correction Factors 42

B Simulation Parameters **43**

C Implementation **43**

C.1 Parallelization 43

C.1.1 What Parallelizes, and Why 43

C.1.2 What Does Not Parallelize 44

C.2 Hardware 44

D Sketch-to-Scene Import Pipeline **44**

D.1 Pipeline 44

D.2 Example Scenes 45

E Resources **45**

1 Introduction

1.1 Motivation

Fluid simulation is used throughout computer graphics, from visual effects to interactive applications, and Smoothed-Particle Hydrodynamics (SPH) is one of the standard ways to produce it [KBST22, RYL+18]: rather than discretizing space into a fixed grid, SPH represents the fluid as a set of particles [GM77, Luc77] that carry their own density, pressure, and velocity, and move with the flow. Every simulation step, each particle needs to know which other particles currently lie within its kernel support in order to compute these quantities – this neighbor search is repeated for every particle, every step, and dominates the total cost of the simulation.

Searching for neighbors by comparing every pair of particles costs $O(n^2)$ time, which becomes prohibitively slow once a scene reaches the millions of particles needed for convincing detail. Acceleration data structures avoid this by only ever comparing a particle against the handful of others that could plausibly be within its support [THM+03, BGT20, FFWL+22], but they differ substantially in how they achieve that – in how they lay particles out in memory, how they are built and rebuilt as particles move, and how well they use the cache and multiple CPU cores [IABT11]. This thesis compares the different algorithms under one shared implementation.

1.2 Goals of the Thesis

The first goal of this thesis is to advance and parallelize the 2D SPH fluid solver built in a previous project [Nah26b] – covering the cubic spline kernel, the core SPH quantities (density, pressure, viscosity, external forces), and particle-based boundary handling – with its per-particle computation now parallelized across CPU cores.

The second goal is to implement several algorithms for a data structure for neighbor search within that solver – a uniform grid combined with index sort, using either linear or Z-curve (Morton) cell indexing and either insertion sort or radix sort to order particles by cell, as well as spatial hashing and compact hashing – so that all variants share the same solver and can be compared fairly against each other.

The third goal is to benchmark these neighbor search algorithms, both in static and dynamic scenes. In static scenes we analyse how the performance and memory consumption scales with the number of particles, while the dynamic scenes allow us to interpret the behaviour of the algorithm in subsequent simulation steps.

1.3 Overview

Section 2 introduces Smoothed-Particle Hydrodynamics: the governing equations, the cubic spline kernel and its discretization, the SPH quantities computed per particle, and the fluid solver used throughout this work.

Section 3 presents the neighbor search acceleration structures compared in this

thesis: the uniform grid and its index sort variants (linear vs. Z-curve indexing, insertion sort vs. radix sort), followed by spatial hashing and compact hashing.

Section 4 discusses how a neighbor search's output – the neighbor lists themselves – can be stored more compactly, reviewing a compression scheme from related work and relating it back to the data structures used in this thesis.

Section 5 presents the performance results for these neighbor search algorithms, first on static, freshly constructed scenes and then inside a dynamic, running simulation, and discusses what the two settings reveal about each algorithm's behavior.

Section 6 summarizes the findings of this thesis and discusses possible directions for future work.

2 SPH Fluid Simulation

Fluid solvers built on an Eulerian grid evaluate density, pressure, and velocity at fixed locations in space [MST04]. Smoothed-Particle Hydrodynamics (SPH) instead takes a Lagrangian view: these same quantities live on discrete particles that themselves are advected with the flow [KBST22].

In Section 2.1 the governing equations for the fluid simulation are introduced. Then in Section 2.2 the concept and derivation of the SPH methods is discussed. Section 2.3 states the quantities computed with SPH and Section 2.4 presents the full fluid solver used in this work.

2.1 Governing Equations

SPH particles are advected with the surrounding flow: a particle's position changes at exactly the local fluid velocity,

$$\frac{d\boldsymbol{x}_i}{dt} = \boldsymbol{v}_i,$$

while that velocity itself is computed according to the Lagrangian form of the Navier-Stokes equation:

$$\frac{d\boldsymbol{v}_i}{dt} = -\frac{1}{\rho}\nabla p_i + \nu\nabla^2\boldsymbol{v}_i + \frac{\boldsymbol{f}_i^{\text{ext}}}{m_i} \tag{1}$$

Equation (1) decomposes the rate of change of a particle's velocity into three contributions. The pressure term pushes a particle away from high-pressure regions and toward low-pressure ones. The viscous term pulls a particle's velocity toward the local average of its neighbors, damping velocity differences and dissipating kinetic energy. The remaining term collects gravity and any other external accelerations acting directly on the particle, like the surface tension introduced in Section 2.3.

Fluid motion as a whole is governed by the Navier-Stokes equations, which rest on two conservation principles: mass and momentum. Conservation of mass forbids fluid from spontaneously appearing or vanishing [RYL+18], while conservation of

momentum ties the evolution of the fluid's velocity field directly to the forces acting upon it [KBST19].

The following section introduces the SPH concept and shows the derivation of the SPH interpolation.

2.2 Smoothed-Particle Hydrodynamics

Smoothed-particle hydrodynamics was introduced independently by Gingold and Monaghan [GM77] and Lucy [Luc77] in 1977 as a mesh-free method for modeling astrophysical phenomena, and has since been adopted widely in computer graphics for simulating fluids and other deformable materials [KBST19, RYL+18]. As has been stated in [Nah26a], the derivation below follows the standard SPH formulation.

SPH reconstructs a field quantity A at a location $\boldsymbol{x}$ by convolving it with the Dirac delta function over the simulation domain Ω [GM82, KBST19]:

$$A(\boldsymbol{x}) = (A * \delta)(\boldsymbol{x}) = \int_{\Omega} A(\boldsymbol{x}')\delta(\boldsymbol{x} - \boldsymbol{x}')d\boldsymbol{x}' \tag{2}$$

By definition, $\delta(\boldsymbol{x})$ vanishes everywhere except at the origin, where it is unbounded, and integrates to one over its domain; in more than one dimension it is simply the product of one-dimensional deltas along each axis. Because δ is a generalized function rather than an ordinary one, it cannot be evaluated directly on a finite set of particles. SPH instead replaces it with a smooth kernel function W_{ij} that has finite support – typically a radius of $2h$:

$$A(\boldsymbol{x}) = (A * W)(\boldsymbol{x}) = \int_{\Omega} A(\boldsymbol{x}')W(\boldsymbol{x} - \boldsymbol{x}', 2h)d\boldsymbol{x}' \tag{3}$$

with h denoting the particle spacing (equivalently, the particle diameter). The kernel adopted throughout this work is the cubic spline [Pri12], and – following the reversed-argument convention used here – takes the form:

$$W_{ij}(2h) = \alpha \begin{cases} (2-q)^3 - 4(1-q)^3 & 0 \leq q < 1 \\ (2-q)^3 & 1 \leq q < 2 \\ 0 & q \geq 2 \end{cases} \tag{4}$$

Here α normalizes the kernel and q is the distance between samples i and j, scaled by h. Integrating the kernel over a disc in two dimensions fixes $\alpha = \frac{5}{14\pi h^2}$.

W_{ij} is used throughout as shorthand for this kernel evaluated with support radius $2h$. A broader discussion of kernel properties and their role in SPH can be found in Liu and Liu [LL10].

Discretizing the kernel-based convolution restricts the sum to neighbors j within particle i's support, and substitutes each particle's volume-mass relation $m_j = \rho_j V_j$ [Pri12], giving the standard SPH interpolant:

$$A_i = \sum_j \frac{m_j}{\rho_j} A_j W_{ij} \tag{5}$$

Evaluating the Navier-Stokes accelerations calls for the spatial derivatives ∇p and $\nabla^2 \boldsymbol{v}$. SPH obtains the gradient of a quantity by shifting the derivative onto the kernel itself:

$$\nabla A_i = \sum_j \frac{m_j}{\rho_j} A_j \nabla W_{ij} \tag{6}$$

This form, however, does not exactly conserve linear and angular momentum. An antisymmetric variant – termed the Hamiltonian form by Price [Pri12] – is used instead:

$$\nabla A_i = \rho_i \sum_j m_j \left(\frac{A_i}{\rho_i^2} + \frac{A_j}{\rho_j^2} \right) \nabla W_{ij} \tag{7}$$

The second derivative could analogously be approximated with the kernel's own Laplacian $\nabla^2 W_{ij}$:

$$\nabla^2 A_i = \sum_j \frac{m_j}{\rho_j} A_j \nabla^2 W_{ij} \tag{8}$$

In practice this Laplacian is numerically unreliable, so SPH implementations instead approximate the second derivative with a finite-difference expression built from the kernel's first derivative alone [Bro85]:

$$\nabla^2 A_i = 2 \sum_j \frac{m_j}{\rho_j} \frac{A_{ij} \cdot \boldsymbol{x}_{ij}}{\boldsymbol{x}_{ij}^2 + 0.01h^2} \nabla W_{ij} \tag{9}$$

with $\boldsymbol{x}_{ij} = \boldsymbol{x}_i - \boldsymbol{x}_j$, $A_{ij} = A_i - A_j$, and a small offset $0.01h^2$ guarding the denominator against division by zero.

Differentiating Equation (4) gives the kernel gradient:

$$\nabla W_{ij} = \alpha \frac{\boldsymbol{x}_i - \boldsymbol{x}_j}{||\boldsymbol{x}_i - \boldsymbol{x}_j|| h} \begin{cases} -3(2-q)^2 + 12(1-q)^2 & 0 \le q < 1 \\ -3(2-q)^2 & 1 \le q < 2 \\ 0 & q \ge 2 \end{cases} \tag{10}$$

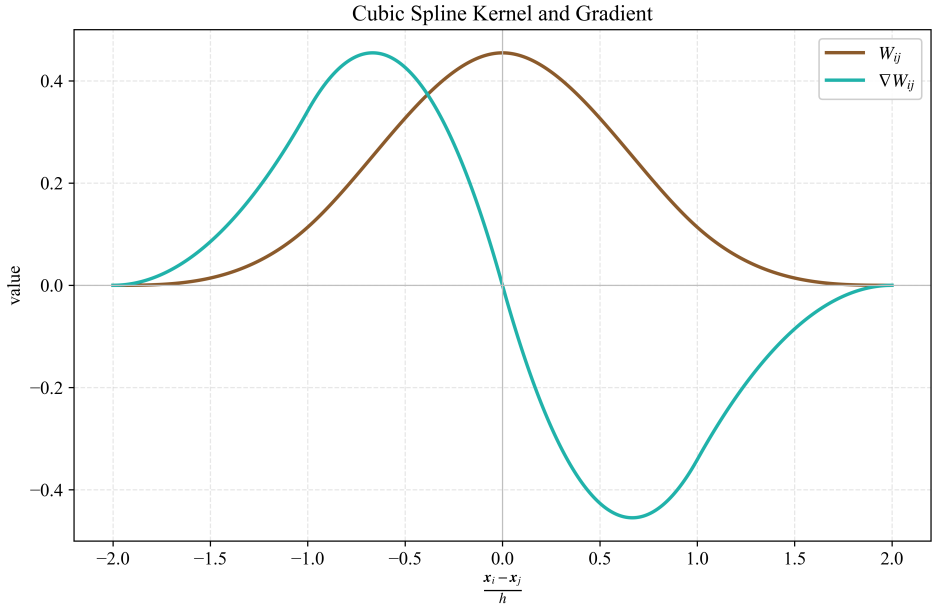
With the kernel and its derivatives established, the following section shows how SPH is put to work in the fluid solver itself.

2.3 SPH Quantities

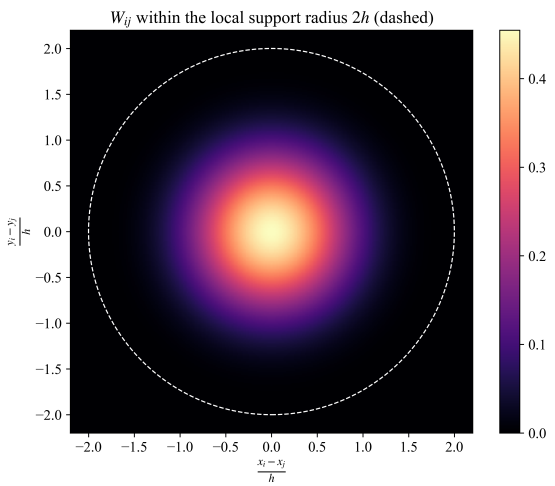
Per particle, SPH requires four quantities: density, pressure acceleration, viscosity acceleration, and any additional acceleration arising from external effects such as rigid-fluid coupling [GPB+19].

Following [Pri12], a particle's density ρ_i is obtained by interpolating the masses of its neighbors j, weighted by the kernel:

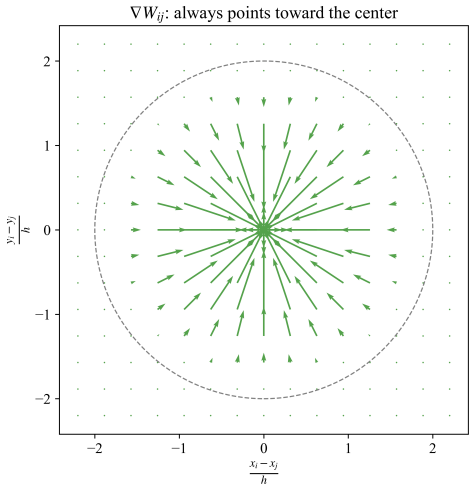
$$\rho_i = \sum_f^{\mathcal{F}} m_f W_{if} + \gamma_1 \sum_b^{\mathcal{B}} m_b W_{ib} \tag{11}$$



(a) W_{ij} and ∇W_{ij} along a single axis through the support radius $2h$.



(b) W_{ij} within the local support radius $2h$.



(c) ∇W_{ij} as a vector field.

Figure 1: The cubic spline kernel and its gradient. (a) W_{ij} is symmetric ($W_{ij}(\boldsymbol{x}_i - \boldsymbol{x}_j) = W_{ij}(\boldsymbol{x}_j - \boldsymbol{x}_i)$), while ∇W_{ij} is antisymmetric ($\nabla W_{ij} = -\nabla W_{ji}$), which is what allows Equation (7) to conserve momentum. Integrating W_{ij} over its local support radius numerically confirms the unity condition to six decimal places. (b) The concentric, radially symmetric contours show that the kernel depends only on the distance $||\boldsymbol{x}_i - \boldsymbol{x}_j||$, not on direction. (c) Every gradient vector points toward the center, since W_{ij} is largest there; a point and its mirror image across the center always have oppositely-pointing vectors, the geometric picture of antisymmetry.

where $\mathcal{F}$ and $\mathcal{B}$ are the sets of fluid and boundary neighbors of particle i respectively, and γ_1 is a fixed correction factor applied to the boundary contribution; see Appendix A for the rationale behind γ_1 and the value used.

Applying the antisymmetric form of Equation (7) to the pressure term of Equation (1) yields the pressure acceleration:

$$-\frac{1}{\rho_i}\nabla p_i = -\sum_f^{\mathcal{F}} m_f \left(\frac{p_i}{\rho_i^2} + \frac{p_f}{\rho_f^2}\right) \nabla W_{if} - \gamma_2 \sum_b^{\mathcal{B}} m_b \left(\frac{p_i}{\rho_i^2} + \frac{p_i}{\rho_b^2}\right) \nabla W_{ib} \tag{12}$$

where, as in Equation (11), the first sum is over the fluid neighbors $\mathcal{F}$ and the second over the boundary neighbors $\mathcal{B}$; γ_2 is a second fixed correction factor, and the boundary term uses p_i in place of the boundary particle's own (undefined) pressure. See Appendix A for the rationale behind this mirroring and the value of γ_2 used.

For the computation of the pressure, different approaches can be used, such as implicit incompressible SPH [ICS+14], its extension to elastic solids [PGBT18], and rigid-fluid coupling via a particle-sampled boundary [AIA+12] – the latter being the basis for the boundary-handling correction factors and pressure mirroring used in this work (see Appendix A). Since this work focuses on data structures for the neighbor search rather than on the pressure solver itself, the pressure is simply computed with the state equation for an explicit density deviation [KBST19, BT07]:

$$p_i = \max\Big(k(\rho_i - \rho_0),\ 0\Big) \tag{13}$$

Here ρ_0 denotes the fluid's rest density and k a user-chosen gas-stiffness coefficient. Equation (13) only produces pressure once a particle's density climbs above ρ_0; a density below rest yields no restoring pressure at all, a well-known limitation of this formulation. Clamped this way, the state equation enforces near-incompressibility in the weakly-compressible sense [KBST19, BT07], with k controlling how strongly compression is resisted.

The viscous term of Equation (1) is likewise discretized with the finite-difference kernel-gradient approximation [KBST22, Bro85]:

$$\nu\nabla^2 \boldsymbol{v}_i = 2\ \nu \sum_j \frac{m_j}{\rho_i}\ \frac{\boldsymbol{x}_{ij}\cdot\boldsymbol{v}_{ij}}{\boldsymbol{x}_{ij}^2 + 0.01\ h^2}\ \nabla W_{ij} \tag{14}$$

Equation (14) pulls each particle toward the velocity of its neighborhood, damping relative motion and driving the local flow toward rest.

Fluid near a free surface – for instance in a splash or a thin sheet – tends to fragment into loose individual particles rather than holding together as a coherent body of liquid. To counteract this, a surface tension acceleration is added that pulls each particle toward the kernel-weighted center of its neighborhood, favoring compact, rounded surfaces over scattered droplets, following the formulation of [BT07]:

$$\boldsymbol{a}_i^{st} = -\frac{\kappa}{m_i}\sum_j m_j W_{ij}\boldsymbol{x}_{ij} \tag{15}$$

Now that the concept of SPH and the SPH quantities have been formally introduced, the next section presents the SPH fluid solver.

2.4 SPH Fluid Solver

In order to compute the SPH quantities, the fluid solver first needs to query the neighbors for each fluid particle i. The construction and the query of the neighbor search is handled in Section 3. After finding the neighbors for each particle i, the density ρ_i and consequently the pressure p_i are computed. This is followed by the acceleration computation: the pressure acceleration, viscosity acceleration, gravity and surface tension (Section 2.3). Eventually, an integration step is made with Euler-Cromer. First the velocity at the consequent time step is computed, followed by the advection of the particles. A whole simulation step is given by Algorithm 1.

Algorithm 1: *SimulationStep*()

foreach *particle* i **do**
  find neighbors j
foreach *particle* i **do**
  $\rho_i(t) \leftarrow$ Eq. (11);
  $p_i(t) \leftarrow$Eq. (13);
foreach *particle* i **do**
  $\boldsymbol{a}_i^p(t) \leftarrow$ Eq. (12);
  $\boldsymbol{a}_i^\nu(t) \leftarrow$ Eq. (14);
  $\boldsymbol{a}_i^g(t) \leftarrow g$;
  $\boldsymbol{a}_i^{st}(t) \leftarrow$ Eq. 15;
  $\boldsymbol{a}_i(t) \leftarrow \boldsymbol{a}_i^p(t) + \boldsymbol{a}_i^\nu(t) + \boldsymbol{a}_i^g(t) + \boldsymbol{a}_i^{st}(t)$;
foreach *particle* i **do**
  $\boldsymbol{v}_i(t + \Delta t) \leftarrow \boldsymbol{v}_i(t) + \Delta t \boldsymbol{a}_i(t)$
  $\boldsymbol{x}_i(t + \Delta t) \leftarrow \boldsymbol{x}_i(t) + \Delta t \boldsymbol{v}_i(t + \Delta t)$

The following scenes in Figures 2, 3, 4 and 5 illustrate the SPH fluid solver in action, showing how the fluid evolves over time in different complex, artist-drawn environments.



(a) Shortly after release.



(b) The wave reaches the far end.



(c) Later in the same run.

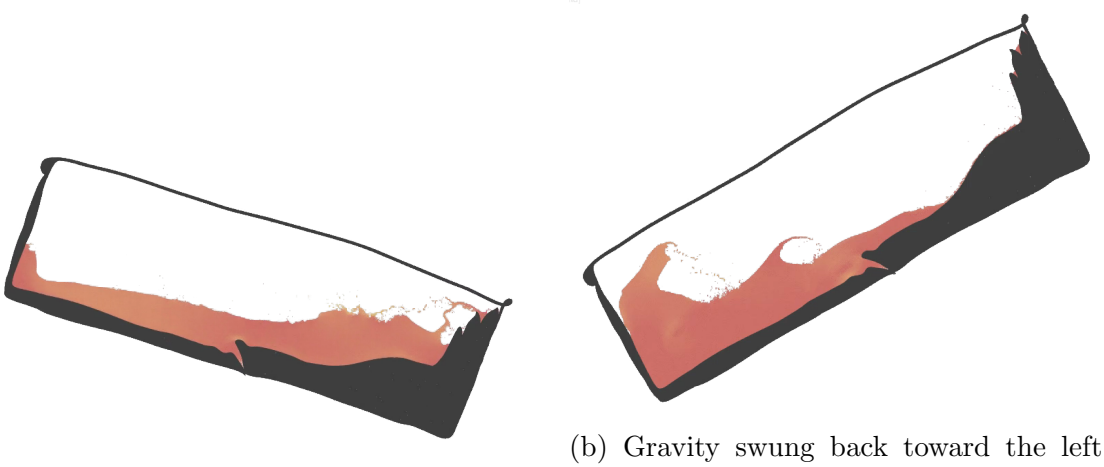
Figure 2: The breaking dam wide scene at three points in the simulation. In (a), the released fluid has already formed a smooth wave sloping down from the left, still short of the container's rounded right end. By (b), the wave has traveled the length of the container and breaks against the far wall, visible as a turbulent splash. In (c), the fluid has leveled out into a gently rippled surface spanning the full width of the container.



(a) Shortly after release.

(b) Later in the same run.

Figure 3: The underground (braided channel) scene at two points in the simulation. In (a), most of the fluid is still pooled near the top of the domain, only beginning to find its way into the braided channel below. By (b), the flow has traced a path down through the braided channels toward the lower chamber, still visibly turbulent where it threads through the narrow passages the sketch's boundary strokes allow.



(a) Gravity tilted toward the right wall.

(b) Gravity swung back toward the left wall.

Figure 4: The tsunami (coastal wave) scene at two points in the same run, with gravity swinging back and forth like a pendulum instead of pointing straight down (see ComputeSwingingGravity in `ParticleSimulation.cpp`). The camera rotates to match gravity's current tilt, so the tank appears to physically rock while gravity itself stays visually "down". In (a), the fluid pools toward the jagged mountain wall on the right; in (b), roughly half a swing period later, gravity has reversed and the fluid instead pools toward the flat wall on the left.



(a) Mid-fall, still turbulent.

(b) Later in the same run.

Figure 5: The downwards (winding channel) scene at two points in the simulation. In (a), the fluid is still falling through the upper bends of the channel, visibly turbulent with a whitewater-like spray pattern where it strikes the walls. In (b), more fluid has collected in the lower pool while the upper channel continues to feed it, showing the solver sustaining a continuous flow through a winding, non-rectangular boundary over a long run.

The following section presents the acceleration data structures used for the neighbor search in SPH.

3 Neighbor Search

Different data structures and algorithms for the neighbor search typically rely on subdividing the space or simulation domain into smaller ones, followed by computing the distance to candidate or potential neighbor [FFWL+22]. Compared to the

naive approach, where the distance among all particle pairs needs to be computed, acceleration data structures significantly reduce the computational costs for the neighbor search [THM+03]. While other data structures, such as the octree introduced by [FFWL+22], bounding volume hierarchies, or other alternatives, are being explored elsewhere, this work focuses on the uniform grid as the underlying data structure and explores different algorithms for constructing and utilizing the data structure.

On top of the uniform grid, two families of algorithms are considered in this work: index sort and hashing. Index sort maps each particle's 2D cell coordinates to a 1D key and sorts the particles by this key, while hashing maps the cell coordinates directly onto a hash table without requiring a sort. [IABT11] propose optimizations for both; index sort with the Morton Z-Indexing and compact hashing.

The following section introduces the uniform grid data structure and explains how it is used for efficient neighbor search in SPH simulations.

3.1 Uniform Grid

The uniform grid spatial data structure is used to subdivide the simulation domain into uniform cells of size d. Each cell is represented by its 2D indices k and l in the x and y directions respectively. The cell indices, in which the particles are located, are computed using their positions and the cell size as follows:

$$\boldsymbol{c} = (k,\ l) = \left(\left\lfloor \frac{x_{i,x}}{d} \right\rfloor, \left\lfloor \frac{x_{i,y}}{d} \right\rfloor \right) \tag{16}$$

The optimal cell size d is equal to the local support radius $2h$ [IABT11]. In this case, only candidate neighbors lying in the same cell and the 8 surrounding cells need to be tested as can be seen in Figure 6. These cells are denoted by

$$(k + \Delta x, l + \Delta y)$$

where

$$\Delta x, \Delta y \in \{-1, 0, 1\}.$$

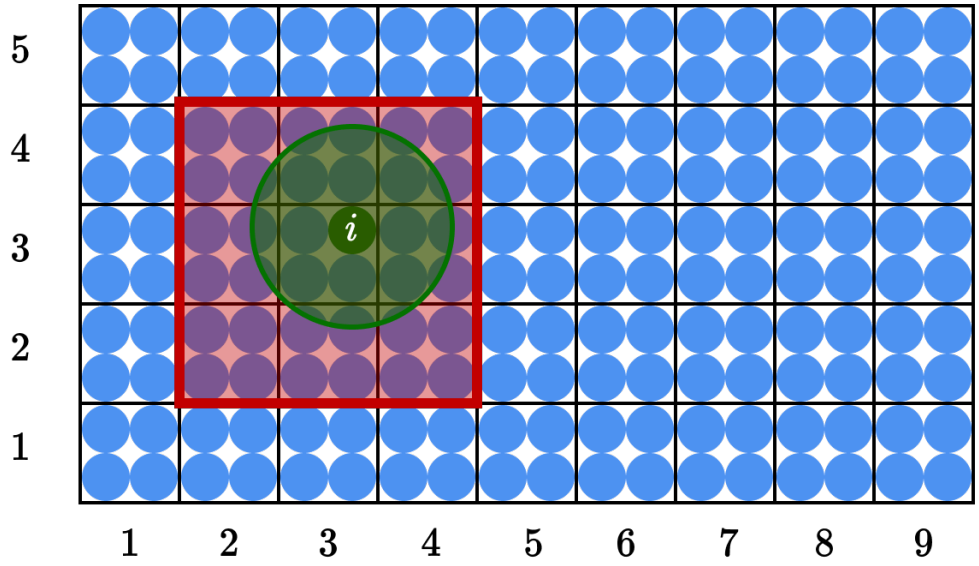


Figure 6: A 9x5 uniform grid with perfectly sampled particles. The grid cell size d is equal to $2h$, making it 4 particles per grid cell. To find the neighbors of particle i which are within the local radius, candidate neighbors lying within the current cell $\boldsymbol{c} = (3, 2)$ and the 8 neighboring cells need to be tested for neighborhood. These 9 cells with $k, l \in \{2, 3, 4\}$ are outlined in the figure.

In Sections 3.2 and 3.3 the index sort and hashing methods for accelerating the neighborhood search using a uniform grid are discussed in detail.

3.2 Index Sort

Index sort is one algorithm that utilizes uniform grid to accelerate the neighborhood search. It consists of two main components: a list of sorted handles H and a list of cell references R, which point to the first particle in each cell.

As can be seen in Figure 7, each handle in H stores the index of a particle and the corresponding cell index as a (*particle index*, *cell index*) pair. The handles in H are sorted by the cell index c. The sorted handles place particles within the same cell close to each other in the list. The position of the first handle in each range in H is the reference stored in R. Non-occupied cells have no reference.

Algorithm 2 describes how index sort is implemented to perform the neighborhood search. The handles are initially empty, so they need to be rebuilt by iterating over all particles, computing their cell indices by the corresponding indexing scheme introduced in Section 3.2.1 and adding them as handles to the list H. This is seen in the first block in Algorithm 2. If the handles are not empty, their cell indices are updated, as shown in the else block. The next step is sorting the handles H by their cell indices. The sorted handles now has all particles within the same cell placed consecutively in the list. Thereafter, the references in R are created by comparing $H[h]$ to $H[h-1]$ for all positions h in H to find the first handle for each cell. Since this check only ever looks at one fixed pair of neighboring positions, every position in H can be checked independently, which is what makes this step parallelizable without needing a single sequential pass that tracks

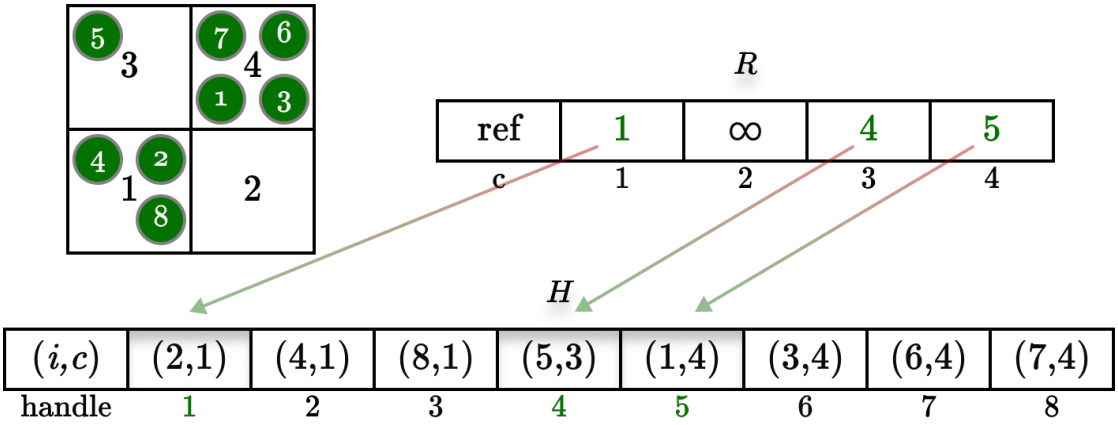


Figure 7: The illustration depicts the construction of index sort using a 4x4 uniform grid and 8 particles. The particles are randomly distributed in the grid cells. Some cells are heavily filled and some are empty, imitating the behavior of a fluid simulation. The handles list H is constructed by iterating over all particles, computing their 1D cell indices (further discussed in 3.2.1) and inserting them into H. Particle 1 is at cell 4, so it is inserted to the handles as $(1, 4)$, particle 2 is at cell 1 so it is inserted as $(2, 1)$ and so on. After iterating over all particles, the handles are sorted by the cell index c (further discussed in 3.2.2). After sorting the handles, the cell references need to be found. This is done by comparing each handle in H to the handle immediately before it in the sorted order: if a handle's cell index differs from its predecessor's – or it is the very first handle in H – it is the first handle for that cell, and its position is written into the references list R. For example, the first handle in H, $(2, 1)$, has no predecessor, so its position is written to R for cell 1. The next handle whose cell index differs from its predecessor is $(5, 3)$, so the reference for cell 3 is set to that handle's position, 4. This check is repeated for every position in H until all handles have been visited. Since the reference list R is initialized with ∞, cells that never appear as a cell index in H remain unchanged and keep the value ∞.

the most recently seen cell. Finally, for each particle, the neighborhood query is performed by checking the neighboring cells using the references in R to find candidate neighbors. If a candidate neighbor is indeed a neighbor, it is updated as a neighbor of the examined particle.

Algorithm 2: *IndexSort*()

```
// Construction (IndexSort::construct)
if H.isEmpty() then
    foreach particle i do
        (k, l) ← (⌊x_{i,x}/d⌋, ⌊x_{i,y}/d⌋);
        c ← ComputeCellIndex(k, l);
        H.insert({i, c});
else
    H.updateCellIndices();
H.sortByCellIndex();
R.FindCellReferences();
// Query (IndexSort::query)
foreach particle i do
    FindNeighbours(i);
```

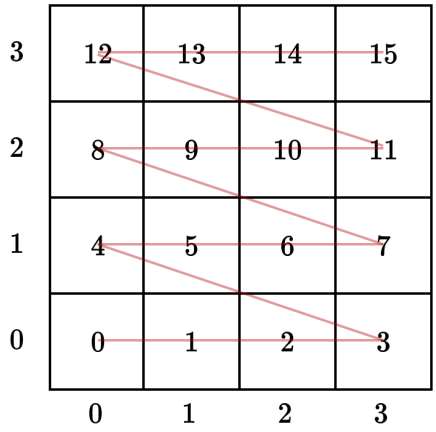
Building or updating the handles, computing cell indices, and finding cell references (via $FindCellReferences$) are each a single pass over the n particles, so they cost $O(n)$; the same holds for the neighborhood query itself, since every particle only tests candidates in a constant number of nearby cells. The sorting step is therefore the only part of index sort whose cost depends on the chosen sorting algorithm, giving an overall complexity of $O(n + \text{sortingCost})$, where sortingCost is $\Theta(n^2)$ for insertion sort or $O(n)$ for radix sort (Section 3.2.2).

3.2.1 Indexing

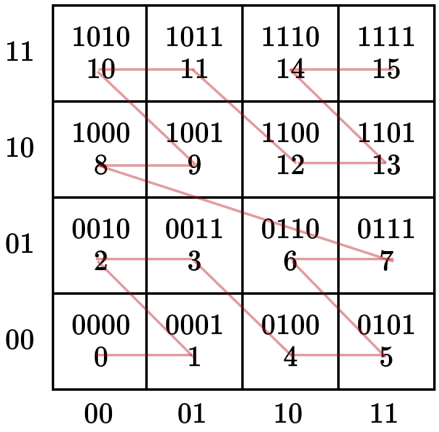
The indexing scheme used in the construction of index sort is important, because after sorting the handles by their cell indices, the spatial locality of the particles is preserved in the 1D array, which facilitates efficient neighbor search. The two indexing schemes are linear indexing and Morton Z-indexing (Figure 8). Throughout this thesis, index sort without further qualification refers to the variant using linear indexing, while the variant using Morton Z-indexing is referred to as Z-index sort.

On the one hand, linear indexing groups cells row-by-row, which preserves spatial locality along the rows but may not be as effective for locality along the columns. On the other hand, Morton Z-Indexing interleaves the bits of the cell indices, which helps to maintain spatial locality in both dimensions [IABT11, Mor66, PF01, LSY01].

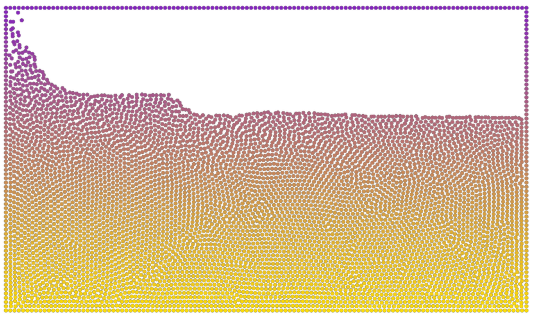
The linear curve maps 2D indices (k, l) to 1D indices c by traversing the grid



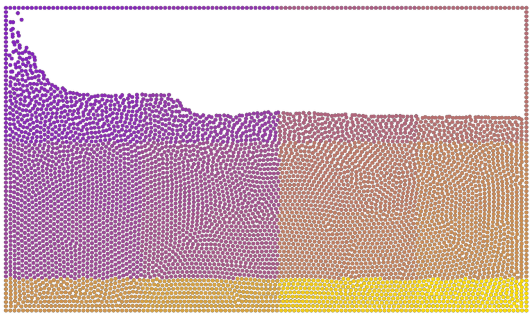
(a) Linear indexing, concept.



(b) Morton Z-indexing, concept.



(c) Linear indexing, scene.



(d) Morton Z-indexing, scene.

Figure 8: Linear indexing vs. Morton Z-indexing for a 4x4 uniform grid, conceptually (a, b) and in an actual particle scene (c, d). In linear indexing, the 1D index is computed as in Equation (17); in Morton Z-indexing, the bits of the cell indices are interleaved to create a single index as in Equation (18).

row-by-row. This mapping is given by:

$$c = k + l \cdot K \tag{17}$$

where K is the number of cells in the x direction. The z-Curve computed by the Morton Code Z-Indexing works by interleaving the bits of the cell indices k and l to create a single index. To interleave the bits, both k and l are spread, l is shifted left by 1 bit, and then the interleaved index is computed by bitwise ORing the spread values. So if k and l were represented in binary as $k = k_m k_{m-1} \dots k_0$ and $l = l_m l_{m-1} \dots l_0$, the interleaved index would be:

$$c = l_m k_m l_{m-1} k_{m-1} \dots l_0 k_0. \tag{18}$$

3.2.2 Sorting

In index sort, the handles are sorted by their cell indices. Different works choose different sorting algorithms for this step. [BGT20], for instance, sort their handles

with a parallelized stable merge sort. Others [OD08,Gre10,GSSP10] parallelize the radix sort. This work however compares two different sorting methods: insertion sort and radix sort, both sequential. (There was no reason for not parallelizing the radix sort, it just wasn't done and still showed efficient computation in practice.)

Insertion sort (Algorithm 3) builds up a sorted list one handle at a time, inserting each new handle into its correct position among the ones already sorted. As a sorting algorithm, its computational complexity is $\Theta(n^2)$ [Knu98]. However, due to the CFL condition (see Appendix B), only few particles change their cell index between two consecutive simulation steps [IABT11]. This fact results in an almost sorted list of handles, which significantly improves the actual computational costs. The results in Section 5.1.1 show the bad side of insertion sort, while the results in Section 5.1.2 show its good side.

Algorithm 3: $InsertionSort(H)$

```
for i ← 1 to |H| − 1 do
    handle ← H[i];
    j ← i − 1;
    while j ≥ 0 ∧ H[j].c > handle.c do
        H[j + 1] ← H[j];
        j ← j − 1;
    H[j + 1] ← handle;
```

Radix sort (Algorithm 4) [Sew54] never compares two handles directly. Instead, it treats the cell index c as D digits of a fixed radix b – D is renamed from the usual d to avoid clashing with the cell size d from Section 3.1 above – and sorts the handles one digit at a time using a stable counting sort, starting with the least-significant digit (LSD radix sort). Digit p of a cell index c is given by

$$digit(c, p) = \left\lfloor \frac{c}{b^p} \right\rfloor \bmod b, \tag{19}$$

so digit 0 is the least-significant and digit $D - 1$ the most-significant. This work uses $b = 2^{16}$, so a 32-bit cell index is split into $D = \lceil 32/16 \rceil = 2$ digits of 16 bits each.

Because it never compares two handles, radix sort is not bound by the $\Omega(n \log n)$ lower bound that applies to comparison-based sorts [Knu98]. Each digit is sorted in $O(n + b)$, so the total cost across D digits is $O(D \cdot (n + b))$, which for a fixed b becomes effectively linear in n – in sharp contrast to insertion sort's $\Theta(n^2)$. This comes at the cost of memory: unlike insertion sort, radix sort is not in-place, requiring an auxiliary array of size n for every pass.

Algorithm 4: *RadixSort*(H)

```
n ← |H|;   aux ← array of n empty handles;
for p ← 0 to D − 1 do
    // Histogram
    count ← array of b zeros;
    foreach handle h ∈ H do
        count[digit(h.c, p)] ← count[digit(h.c, p)] + 1;
    // Prefix sum
    for k ← 1 to b − 1 do
        count[k] ← count[k] + count[k − 1];
    // Stable scatter, in reverse
    for i ← n − 1 to 0 do
        count[digit(H[i].c, p)] ← count[digit(H[i].c, p)] − 1;
        aux[ count[digit(H[i].c, p)] ] ← H[i];
    H ↔ aux ;                          // swap roles for the next digit
```

Next, the hashing methods are discussed in detail.

3.3 Hashing

Hashing maps a particle's cell indices (k, l) directly onto a hash table, without the sorting step index sort requires. The two variants considered in this work – spatial hashing and compact hashing – share the same construction and query procedure, given in Algorithm 5: the table T is cleared, every particle is hashed into its own cell's table entry, and then, for every particle, its own cell and the 8 surrounding cells are hashed and their entries checked for candidate neighbors. The two variants differ only in the hash function $\phi(k, l)$ and in what kind of table T that function indexes into, described in Sections 3.3.1 and 3.3.2 respectively.

3.3.1 Spatial Hashing

Spatial hashing maps all cells in the simulation domain onto a table T of fixed size m, using

$$\phi_{spatial}(k, l) = (k \cdot p_1 \oplus l \cdot p_2) \bmod m \tag{20}$$

where p_1 and p_2 are two large prime numbers and $\oplus$ is the bitwise XOR operation [THM+03]. The smaller the table, the more hash collisions occur; the larger it is, the fewer. An appropriate table size, trading memory for speed [KBST19, THM+03, IABT11], is two times the number of particles n – this work rounds that up to the next prime, $m = \text{nextPrime}(2n)$, a common choice to spread hash values more evenly across the table.

Some prior work [THM+03] additionally reserves k fixed slots for every one of the m cells on initialization, giving $O(m \cdot k)$ memory regardless of how many cells are actually occupied. This implementation does not: each of the m table entries is instead a dynamically-growing list, so a cell only uses memory proportional to the particles actually hashed into it. Figure 9 illustrates the resulting table.

Algorithm 5: *Hashing*(T)

```
// Construction (construct)
T.clear();
foreach particle i do
    (k, l) ← (⌊x_{i,x}/d⌋, ⌊x_{i,y}/d⌋);
    T[φ(k, l)].insert(i);
// Query (query)
foreach particle i do
    (k, l) ← (⌊x_{i,x}/d⌋, ⌊x_{i,y}/d⌋);
    foreach Δx ∈ {−1, 0, 1} do
        foreach Δy ∈ {−1, 0, 1} do
            foreach candidate j ∈ T[φ(k + Δx, l + Δy)] do
                if ‖x_j − x_i‖ ≤ 2h then
                    insert j into neighbor set of i;
```

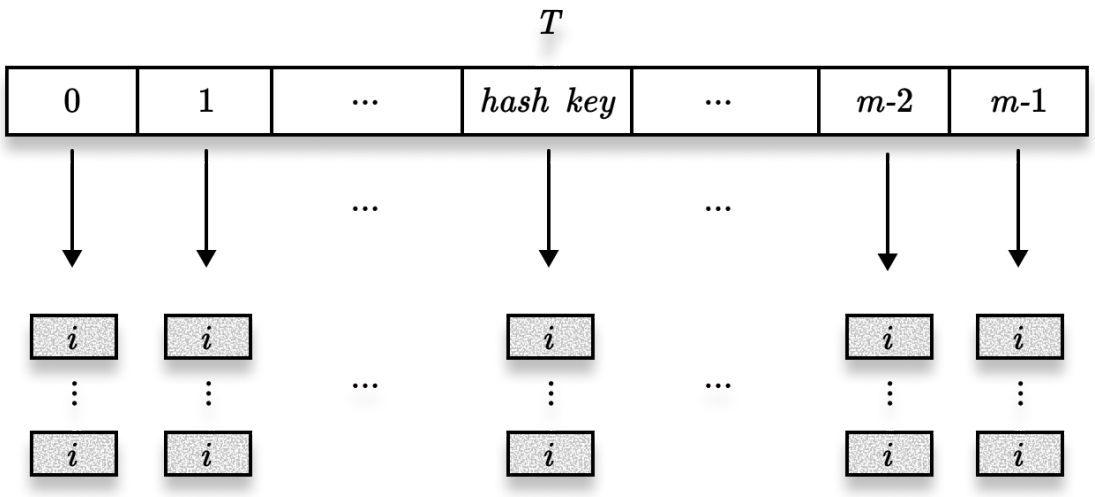


Figure 9: Hash table T is of size m. Each grid cell is hashed to a slot at $\phi_{spatial}(k, l)$ and particles i in the same cell are hashed to the same slot.

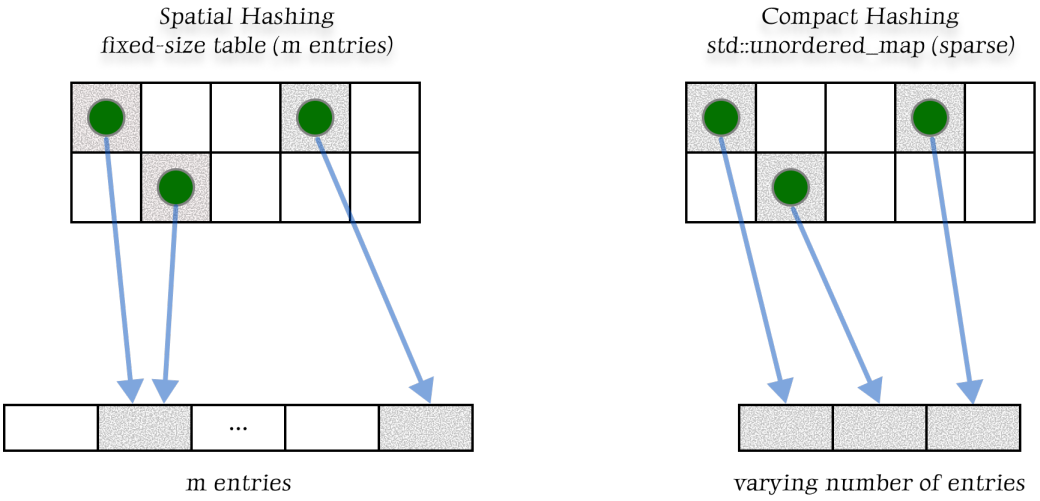


Figure 10: Comparison between spatial and compact hashing. Spatial hashing uses a fixed-size hash table, while compact hashing dynamically allocates memory for the hash table.

3.3.2 Compact Hashing

Compact hashing's advantage over spatial hashing is reduced memory consumption: only occupied cells ever appear in the table, instead of reserving space for the whole simulation domain up front, as can be seen in the comparison in Figure 10. Its hash function omits the table size m entirely,

$$\phi_{compact}(k, l) = k \cdot p_1 \oplus l \cdot p_2, \tag{21}$$

and is used directly as the key into a table T that only stores entries for cells which currently contain at least one particle – an empty cell has no entry at all, rather than an entry that happens to be empty. Consequently, the memory used scales with the number of occupied cells, not with the size of the simulation domain.

This implementation rebuilds T from scratch every simulation step ($T.clear()$ in Algorithm 5, followed by re-inserting every particle), rather than incrementally adding and removing individual cells as particles cross cell boundaries. [IABT11] additionally split their compact hash table into a fixed-size handle array pointing into a separate, smaller list of only the occupied cells, bounding the table's own memory to $O(m+n\cdot k)$; this implementation does not use that indirection, relying instead directly on a hash map keyed by $\phi_{compact}(k, l)$.

4 Neighbor Representation

4.1 Compression

Storing neighbor lists for each particle takes real memory, and at large particle counts this can become the actual limit on how big a scene fits on a given machine, independent of how fast the neighbor search itself runs. [BGT20] address this

directly with a compression scheme for neighbor lists: instead of storing each neighbor's index directly, they store the difference to the previous neighbor's index in the sorted array (this array corresponds to the handles H introduced in Section 3.2). Most of these differences are very small (often 0 or 1, since particles in the same cell end up next to each other in the sorted array), so they encode 0 and 1 using only control bits, with no extra data byte at all, and fall back to a small number of data bytes only for the rare larger gaps. This brings the average cost down from 4 bytes per neighbor to well under 1 byte with only a small computational overhead for encoding and decoding.

Figure 11 works through this process for a single particle with four neighbors. Each raw neighbor index costs 4 bytes, 16 bytes total. Mapped to their ranks in the cell-sorted handle array H and sorted ascending, three of the four consecutive gaps are small ($\delta = 1$, $\delta = 0$, $\delta = 0$); only the jump from rank 1 to rank 6 needs an explicit data byte ($\delta = 4$). Deltas of 0 or 1 are encoded as a control code alone; $\delta = 4$ falls in the common case of a delta under 256, encoded as a control code plus one data byte (a fourth, 4-data-byte code exists for the rare delta of 256 or more, not needed in this example). This brings the example down to 5 bytes – a 69% reduction, conforming with the 57.5% average measured in Section 5.2.1 across full scenes.

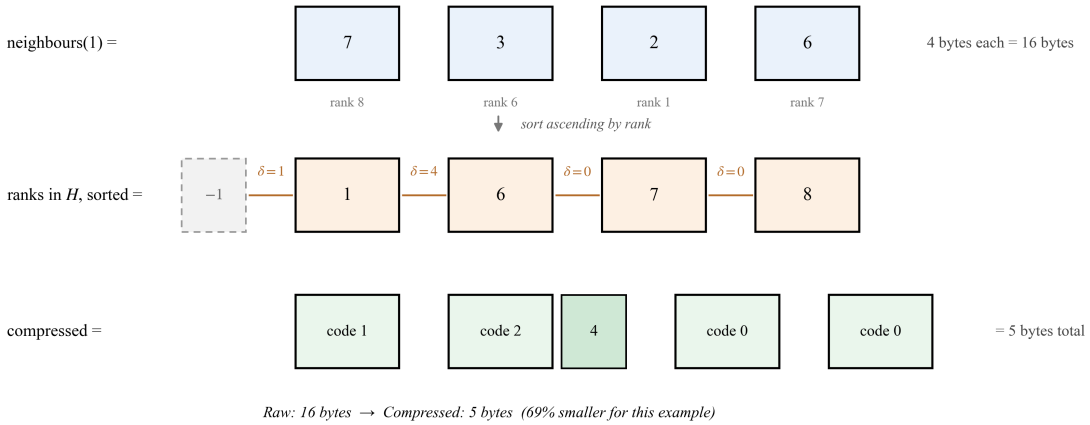


Figure 11: Delta-encoding a single particle's raw neighbor list (top, 4 bytes per index) into its compressed form (bottom): neighbor indices are mapped to their rank in the cell-sorted handle array H, sorted ascending (middle), and the gap δ between consecutive ranks is encoded as a control code alone when $\delta \in \{0, 1\}$, or a control code plus one data byte for the case of $2 \leq \delta < 256$.

[BGT20] also propose a cell-linked-list variant, comparable to compact hashing (Section 3.3.2) but storing only one reference per non-empty cell rather than one per particle – the same idea as the cell references R in Section 3, applied to a sparse list of occupied cells only. This uses about 9× less memory than compact hashing and is faster to build and query, though it was not implemented in this work.

Applying this scheme requires each neighbor to be encoded as its *rank* in a globally cell-sorted ordering, not its raw particle index – otherwise consecutive

entries are not close in memory and the deltas are not small. Index sort already produces this ordering as a side effect of construction (Section 3), so its internal sorted position is already the rank the scheme needs. Spatial and compact hashing do not: particles within one entry are only ordered by insertion, with no spatial meaning, making index sort the natural fit for this compression in this work.

The implementation builds each particle's compressed list from its existing (uncompressed) neighbor list after a query, as shown in Algorithm 6: every neighbor is mapped to its rank via a reverse lookup, the ranks are sorted, and delta-encoded as above. The goal here is to measure the memory saved, not to make this the live representation. One simplification was made relative to [BGT20]: each control code gets its own byte rather than four packed into one, simpler to implement at the cost of some compression. Section 5.2.1 measures the resulting ratio and its effect on total memory.

Algorithm 6: $Compress(neighbors, rankOfParticle)$

```
ranks ← ∅;
foreach particleIndex ∈ neighbors do
    ranks.append(rankOfParticle[particleIndex]);
ranks.sort();
compressed ← ∅; previousRank ← −1;
foreach rank ∈ ranks do
    δ ← rank − previousRank − 1;
    if δ = 0 then
        compressed.append(code 0);
    else if δ = 1 then
        compressed.append(code 1);
    else if δ < 256 then
        compressed.append(code 2, δ);
    else
        compressed.append(code 3, δ);
    previousRank ← rank;
```

5 Results

The following section presents the analysis of the previously introduced algorithms for neighbor search and for neighbour compression. For the neighbour search in Section 3, both single-frames and dynamic scenes are considered. For the neighbor representation and compression in Section 4, the focus is on how well the compression methods reduce memory usage while maintaining efficient access to neighbor information.

5.1 Neighbor Search Analysis

As the neighbor search is a fundamental step in particle-based simulations, its efficiency directly impacts the overall performance of the simulation.

In the static scene analysis, we measure how the construction and query times scale with the number of particles, isolating the neighbor search from other simulation components. Moreover, since the claim behind using compact hashing is that it reduces memory usage while maintaining efficient access, we compare its memory consumption with spatial hashing.

In the dynamic scenes analysis, we evaluate how the algorithms perform when particles move and the neighbor search must be updated frequently.

5.1.1 Static Scenes

The first set of measurements looks at each neighbor search algorithm in isolation, outside of a running simulation. For every measurement, a fresh square scene is created, the acceleration structure is built once, and a single neighbor query is performed – nothing moves, and nothing is repeated. This "static" setting removes any effect that particle motion or accumulated state might have on timing, so the only thing being measured is how construction and query cost scale with the number of particles n in a single frame.

Every static scene has the same shape, shown in Figure 12: a square block of fluid particles arranged on a regular grid, with two layers of boundary particles. The number of particles n varies from one measurement to another. It ranges from 1 million to 80 million. For each n the fluid block is resized to $\sqrt{n}$ particles per side at a fixed particle spacing h, so that only the number of particles changes between runs.

The algorithms measured in these static scenes are the four different variants of index sort, along with spatial and compact hashing. Figure 13 shows a comparison of the radix-sorted variants against the hashing methods. Figure 14 shows the breakdown of construction and query times for linear and Z-curve indexing with radix sort. Figure 15 shows index sort with insertion and Figure 16 index sort and Z-index sort with insertion. Figure 17 compares the memory usage of the hashing methods and eventually Figure 18 shows how memory usage scales with the number of particles.

As can be seen in Figure 13, both radix-sorted index sort variants stay close together across the whole range and clearly outperform the two hashing methods, while they all scale almost linearly with the number of particles. Z-curve indexing is, if anything, slightly faster than linear indexing here, which is the opposite of what happens with insertion sort (Figure 16). This is not because Z-curve is cheaper to construct with radix sort – Figure 14 shows its construction phase is consistently a little more expensive than linear's across the whole 1 to 30 million range, the cost of computing the bit-interleaved index – but its better two-dimensional spatial locality makes the query phase consistently cheaper instead [IABT11], and that saving outweighs the extra construction cost. Between the two hashing methods, spatial hashing is consistently faster than compact hashing despite needing more memory (Figure 17) – so compact hashing's memory saving in this codebase



Figure 12: The static scene used for every measurement in the static setting: a square block of fluid particles (blue) filling the simulation domain, bounded by two layers of boundary particles. Only the particle count n changes between runs; the domain is rescaled with it so that particle spacing, and therefore fluid density, stays constant.

comes at a real cost in access speed, not for free as its name might suggest. This is concentrated in construction rather than query: unlike spatial hashing's fixed-size table, compact hashing's `unordered_map` has to allocate a new node for every newly-seen cell and rehash its bucket array as it grows, and this happens while serialized behind the same critical section that protects the parallel insertion.

Insertion sort's cost depends on how pre-sorted its input already is: it only has to shift an element past however many already-placed entries are larger than it, so nearly-sorted input costs far less than randomly-ordered input, which costs up to $O(n^2)$. Particles' positions are computed row by row, but since each cell spans a 2×2 block of particles (the cell size is twice the particle spacing), two consecutive particle-rows always fall into the same cell-row: the first row's cell indices climb monotonically, and the second row then repeats that same climb from the start, immediately after it. This is not the same as being globally sorted – inserting the second row's low cell indices right after the first row's high ones still costs real shifting work, and this repeats every other row across the whole grid – but it is far closer to sorted than a random ordering, since the disorder is confined to a small, predictable reset rather than spread across the whole array. This matches the measured growth reasonably well: total time grows roughly as $n^{1.5}$ from 1 to 10 million particles (a 31× increase in time for a 10× increase in particles, Figure 15) – well below insertion sort's $O(n^2)$ worst case, but clearly above its $O(n)$ best case.

Even at this reduced cost, insertion sort is impractical in absolute terms: at 10

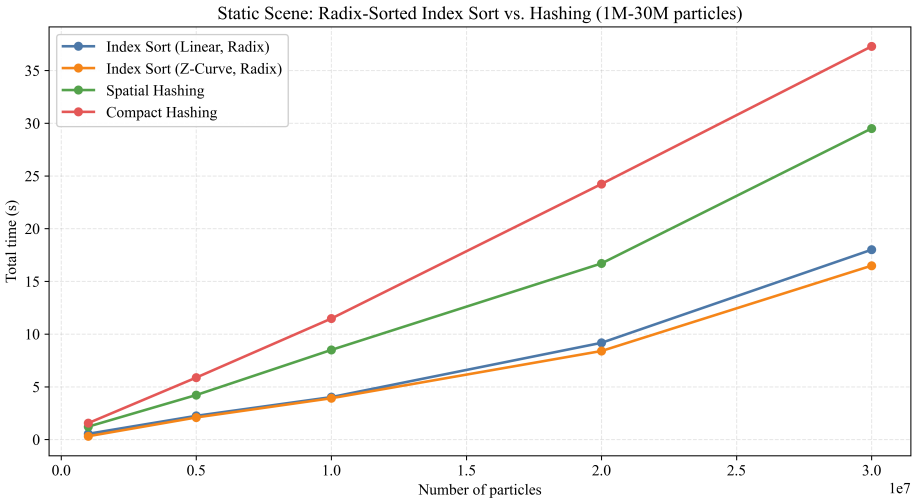


Figure 13: Total time per particle count (construct and query combined) for the four algorithms whose cost stays within a comparable range: index sort with radix sort (linear and Z-curve indexing) and spatial and compact hashing, from 1 to 30 million particles. Insertion sort is excluded here and shown separately in Figure 15, since its cost is orders of magnitude larger and would make the other three curves indistinguishable from zero on the same linear axis. Both index sort variants use radix sort, spatial and compact hashing scale almost linearly with the number of particles, matching the theoretical expectations of their respective computational complexities (Sections 3.2 and 3.3). Z-curve indexing is, if anything, slightly faster than linear indexing here; between the two hashing methods, spatial hashing is consistently faster than compact hashing despite needing more memory (Figure 17). The slight uptick in total time at 30 million particles, a departure from the otherwise linear trend it had been following, is likely attributable to memory pressure, as discussed alongside Figure 18.

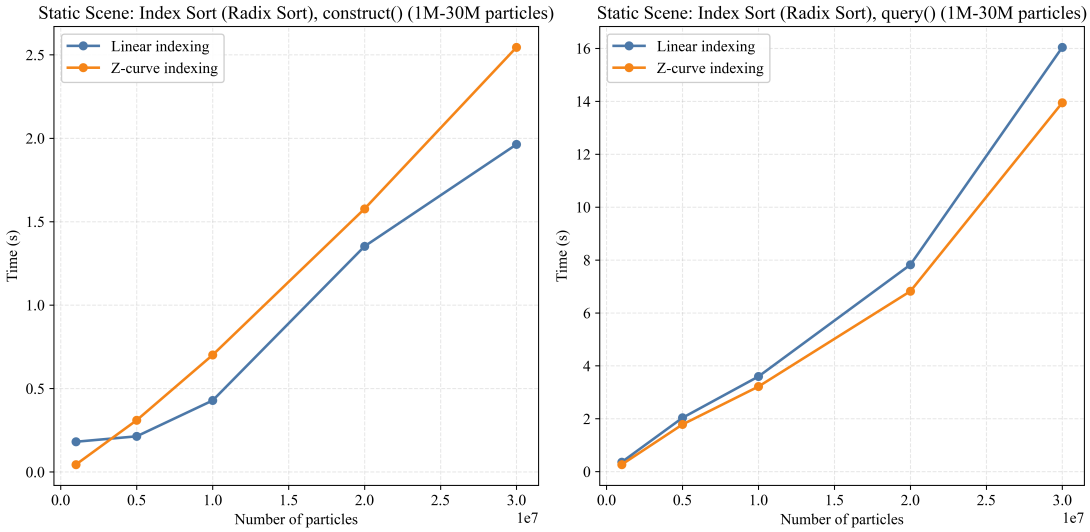


Figure 14: The total time from Figure 13 split into its construct and query halves for the two radix-sorted index sort variants, from 1 to 30 million particles. Z-curve indexing's construct is consistently a little more expensive than linear's, but its query is consistently cheaper overall. This better query time is due to the improved spatial locality of Z-curve indexing.

million particles it takes 430.6 s, compared to 4.0 s for the same linear indexing with radix sort. This is why it is excluded from Figure 13 and shown separately here instead.

Z-curve indexing has no such local structure to exploit: its bit-interleaved index jumps around from one particle to the next with no relationship to the row-by-row generation order, so the handles are close to fully randomly ordered relative to it, and insertion sort is pushed toward its actual worst case (as seen in Figure 16). This is specific to insertion sort – radix sort's cost does not depend on how sorted its input already is, which is exactly why Z-curve indexing no longer looks expensive once radix sort is used instead (Figure 13).

Having covered running time, the remainder of this section turns to memory usage, starting with the two hashing methods, whose trade-off was already anticipated above: spatial hashing was faster but needs more memory. Each hashing structure's memory footprint is computed analytically from its internal containers: the hash table's capacity times the size of an empty `std::vector`, plus the heap-allocated capacity of every cell's neighbor-index vector, for spatial hashing; the `unordered_map`'s bucket array plus one node per occupied cell plus each occupied cell's vector capacity, for compact hashing. This value is deterministic and independent of the machine's available RAM or swap behavior.

Figure 17 shows the result at 1, 5, 10, 20, and 30 million particles. Spatial hashing consistently uses roughly 2.7 times more memory than compact hashing at every scale (1490 MB vs. 559 MB at 30 million particles), which follows directly from the two data structures: spatial hashing allocates a fixed-size table of $m \approx 2n$ cells up front, so even a largely empty cell still costs the overhead of an empty `std::vector`, whereas compact hashing's `unordered_map` only allocates an entry

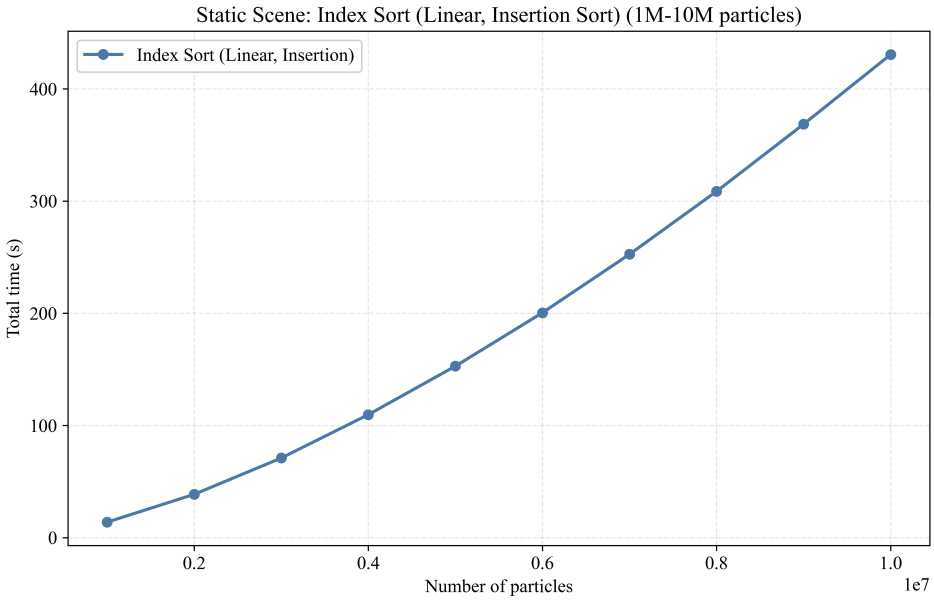


Figure 15: Total time for index sort with linear indexing and insertion sort, from 1 to 10 million particles. Growth is superlinear – roughly $n^{1.5}$ (a 31× time increase for a 10× particle increase) – worse than insertion sort's $O(n)$ best case but better than its $O(n^2)$ worst case, since particles' row-by-row generation order leaves the handles partially, not fully, unsorted. Even so, this is impractical in absolute terms: 430.6 s at 10 million particles, vs. 4.0 s for the same linear indexing with radix sort (Figure 13).

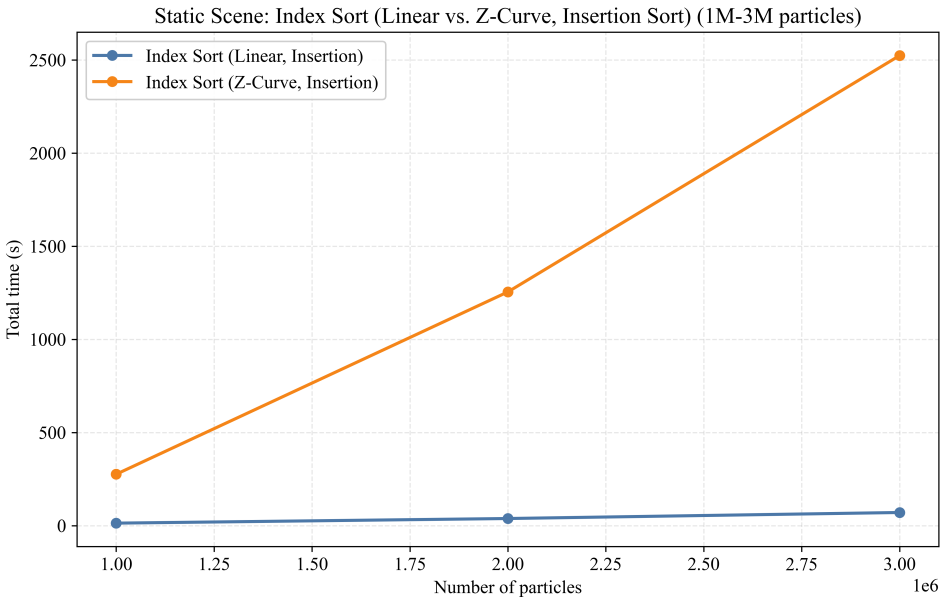


Figure 16: Total time for index sort with insertion sort, linear vs. Z-curve indexing, from 1 to 3 million particles (Z-curve indexing with insertion sort was excluded from the rest of the sweep due to its cost - see the main text). Z-curve indexing is substantially more expensive than linear indexing in this cold, single-shot construction setting: linear indexing happens to correlate with the raster-scan order particles are generated in (Figure 15), while Z-curve's bit-interleaved index does not, leaving its handles close to fully randomly ordered and pushing insertion sort towards its actual $O(n^2)$ worst case.

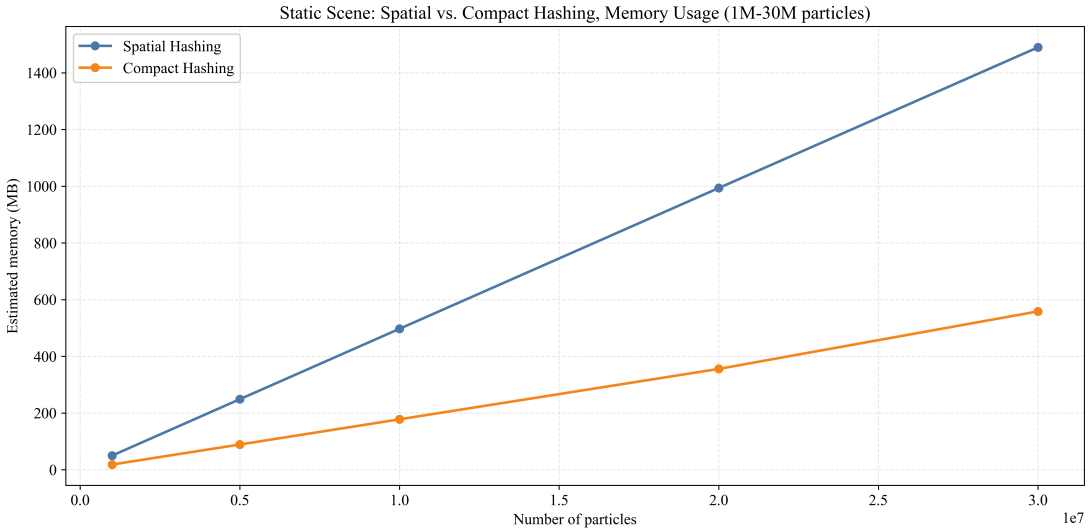


Figure 17: Estimated memory usage of spatial hashing vs. compact hashing at 1, 5, 10, 20, and 30 million particles, computed analytically from each structure's own container capacities rather than measured via `ru_maxrss`, to avoid the swap-related measurement pitfall described above at these particle counts. Spatial hashing consistently needs roughly 2.7 times as much memory as compact hashing (1490 MB vs. 559 MB at 30 million particles), since its fixed-size table of $m \approx 2n$ cells costs the overhead of an empty `std::vector` even for a largely empty cell, whereas compact hashing's `unordered_map` only allocates an entry for cells that actually contain a particle.

for cells that actually contain a particle.

A second sweep repeats the same static-scene methodology up to 80 million particles, restricted to index sort with radix sort (linear and Z-curve indexing), since the other variants are already orders of magnitude slower at this scale. From around 36 million particles onward, the simulation's real memory usage exceeds the machine's 8 GB of RAM (Figure 18), reaching 17.3 GB by 80 million particles as the operating system falls back on memory compression and swap to keep it running.

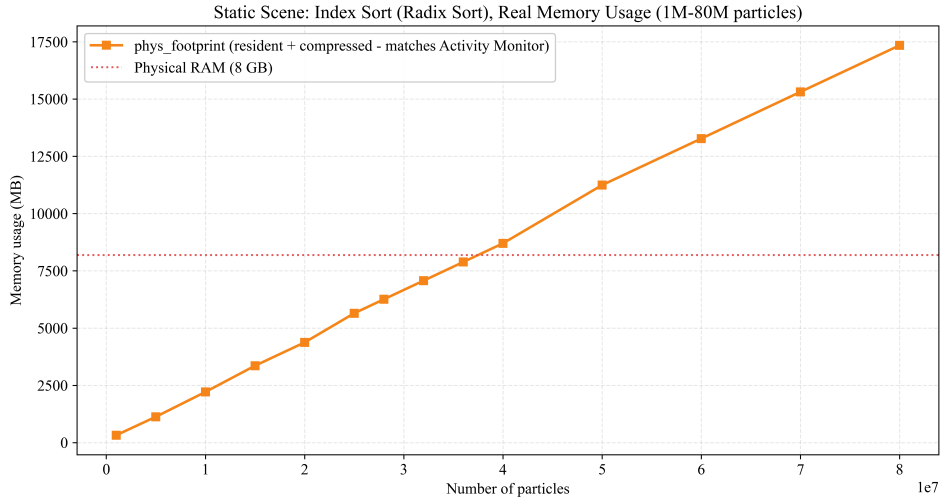


Figure 18: Real memory usage (`phys_footprint`: resident and compressed pages combined) for index sort with radix sort, linear and Z-curve indexing combined into a single process-wide peak per particle count, from 1 to 80 million particles. The dotted line marks the machine's 8 GB of physical RAM, crossed at roughly 36 million particles; by 80 million particles the real footprint reaches 17.3 GB, absorbed by memory compression and swap rather than a hard failure. Even though the 36 million particle is the upper limit of the machine's physical RAM, static scenes with 30 million particles also suffer from memory pressure.

5.1.2 Dynamic Scenes

The previous static measurements time a single, cold construct-and-query call, with no history behind it. A real simulation is not like that: it constructs the data structure and queries the neighbors in every simulation step. This "dynamic" setting ranks the algorithms differently from the static one: Z-curve indexing with insertion sort, for instance, is far more expensive than linear indexing on a cold build (Figure 16), yet cheaper on average once it is running (Figure 24), because most steps only need to fix up the handful of particles that actually crossed a cell boundary.

All six algorithm variants – index sort with linear or Z-curve indexing, each paired with insertion or radix sort, plus spatial and compact hashing – are measured on the same running simulation: the wide-low breaking dam scene (1M fluid particles), run for 1500 simulation steps. Construct time and query time are logged separately for every step, but the figures below mostly plot construct time on its own, since that is where insertion sort's sensitivity to particle motion actually shows up; Figure 24 is the exception, folding both back into a single combined mean per algorithm at the end of this subsection.

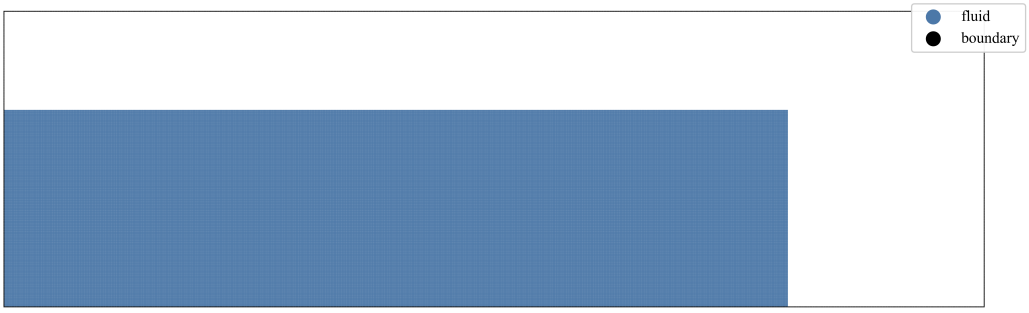


Figure 19: The wide-low breaking dam scene used for every measurement in this subsection, at step 0: a wide, low column of fluid (blue) held back by nothing on its right side, so it collapses under gravity as the simulation runs – unlike the static scenes, particle motion (and therefore neighbor-cell membership) is the whole point here.

Figure 20 compares index sort's construct time with insertion sort, linear vs. Z-curve indexing. Figure 21 checks that construct time actually tracks particle motion rather than measurement noise. Figure 22 repeats the linear vs. Z-curve comparison with radix sort instead of insertion sort. Figure 23 compares spatial and compact hashing. Figure 24 summarizes the mean cost of all six variants over the full run.

For insertion-sorted index sort with both indexing schemes plotted in Figure 20, both curves spike at the same steps, since both are timing the same underlying physics. Z-curve's early spikes (steps 100-450) are taller, while linear's spikes are taller for the remainder of the run. Now at the first glance this looks like noise data from the scene. However, Figure 21 shows how these spikes relate to particle motion. The upper plot belongs to index sort, showing the same spikes seen in Figure 20. These spikes are almost identical to the number of handles changing cells in that simulation step, connecting the fluid's flow motion to the computational time of insertion sort.

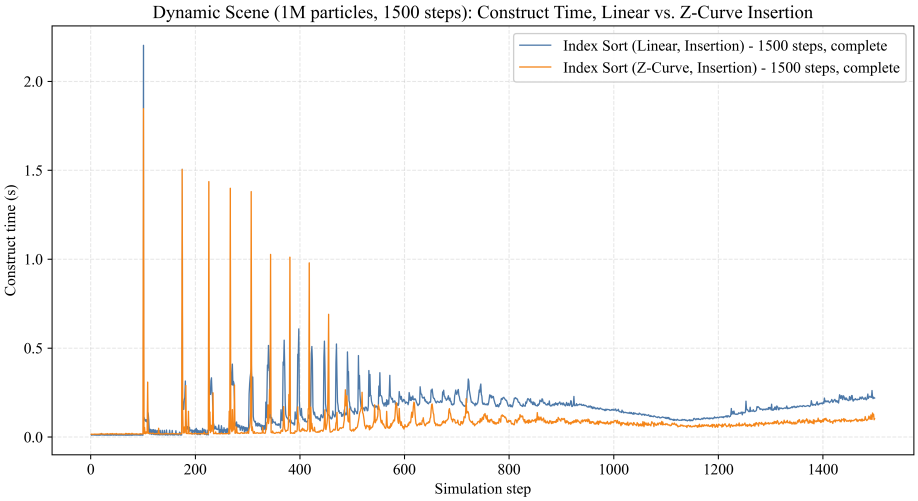


Figure 20: Index sort construct time per step, linear vs. Z-curve indexing, both with insertion sort. Both curves spike at the same steps, since both are timing the same underlying physics – particles crossing cell boundaries – but Z-curve's early spikes (steps 100-450) are taller, while linear's are taller for the remainder of the run.

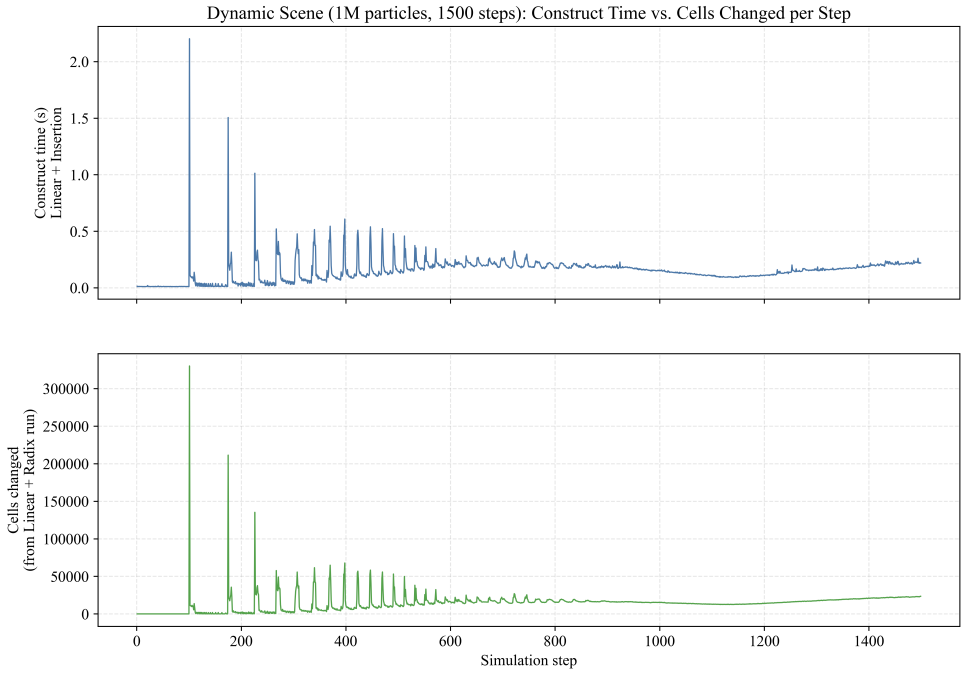


Figure 21: Construct time per step (top, linear indexing with insertion sort) against the number of handles whose cell index changed that step (bottom, logged from the linear indexing with radix sort run of the same scene). The two curves match almost exactly, confirming that the spikes in Figure 20 reflect genuine particle motion between cells rather than measurement noise.

In Figure 22, covering the radix-sorted index sort (also using both indexing schemes) neither curve shows the spikes seen in Figure 20, since radix sort's cost – unlike insertion sort's – does not depend on how sorted the handles already are.

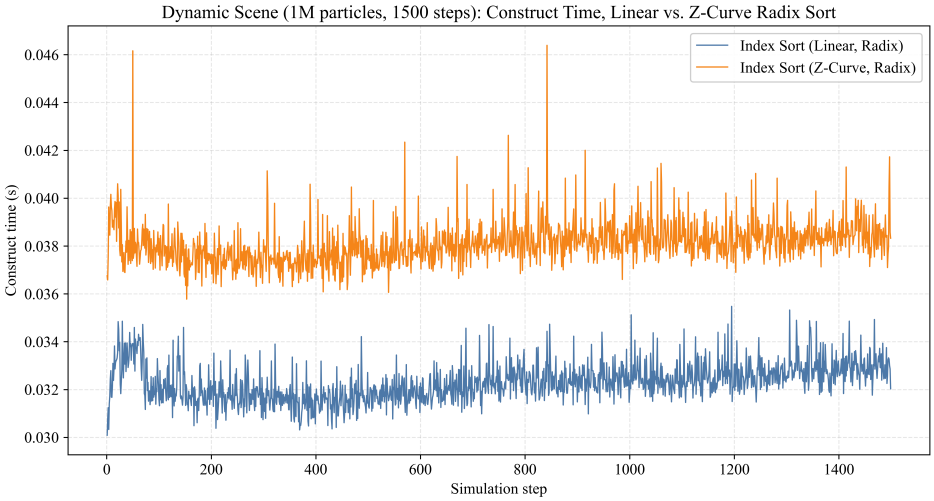


Figure 22: Index sort construct time per step, linear vs. Z-curve indexing, both with radix sort. Unlike insertion sort (Figure 20), neither curve spikes, since radix sort's cost does not depend on how sorted the handles already are.

In Figure 23, both spatial and compact hashing are noisy throughout the run, since neither has an incremental update path and both rebuild their data structure from scratch every step. Compact hashing has one severe, isolated spike around step 1350-1400 that spatial hashing does not share, plausibly caused by an internal hash table resize as the number of occupied cells grows.

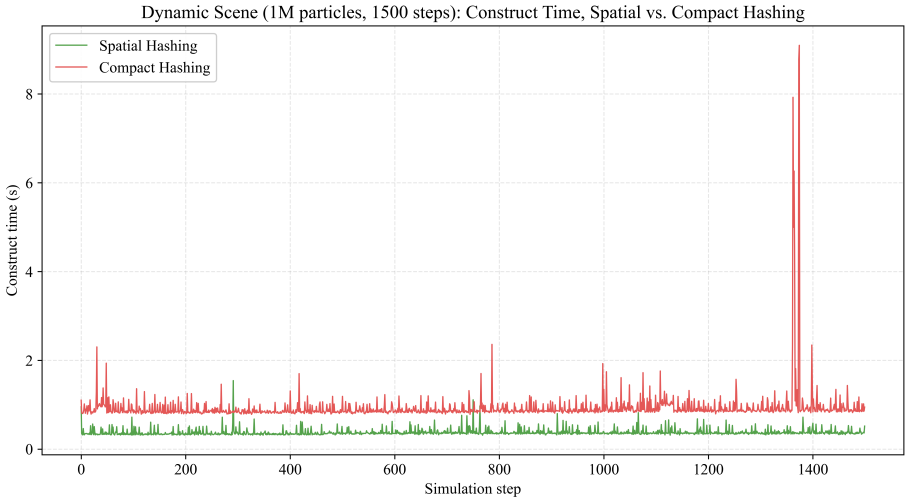


Figure 23: Construct time per step for spatial and compact hashing. Both are noisy throughout the run, since neither has an incremental update path and both rebuild their data structure from scratch every step. Compact hashing's one severe, isolated spike around step 1350-1400 is not shared by spatial hashing, plausibly caused by an internal hash table resize as the number of occupied cells grows.

As Figure 24 shows, all four index sort variants are cheaper than either hashing method. Notably, Z-curve with insertion sort is cheaper on average than linear with insertion sort, despite its larger individual spikes, since its advantage over the remainder of the run outweighs its rougher start.

This ranking is driven by where each family of algorithms spends its time. The construct-time figures above only tell half the story, since query time is logged every step as well but not plotted separately: for all four index sort variants, query – not construct – makes up the majority of the per-step cost once the simulation is past its first, cold step (63-87% of the total, averaged over the run), because the handles are already sorted from the previous step and only need local fix-ups. Spatial and compact hashing invert this: construct dominates instead (56% and 71% of the total respectively), since both discard their table and rebuild it from every particle's position from scratch each step, with no incremental path to fall back on. This is also why the hashing methods' construct-time curves in Figure 23 look noisy throughout rather than settling down the way index sort's does after its first step: there is no "warm" state left to settle into.

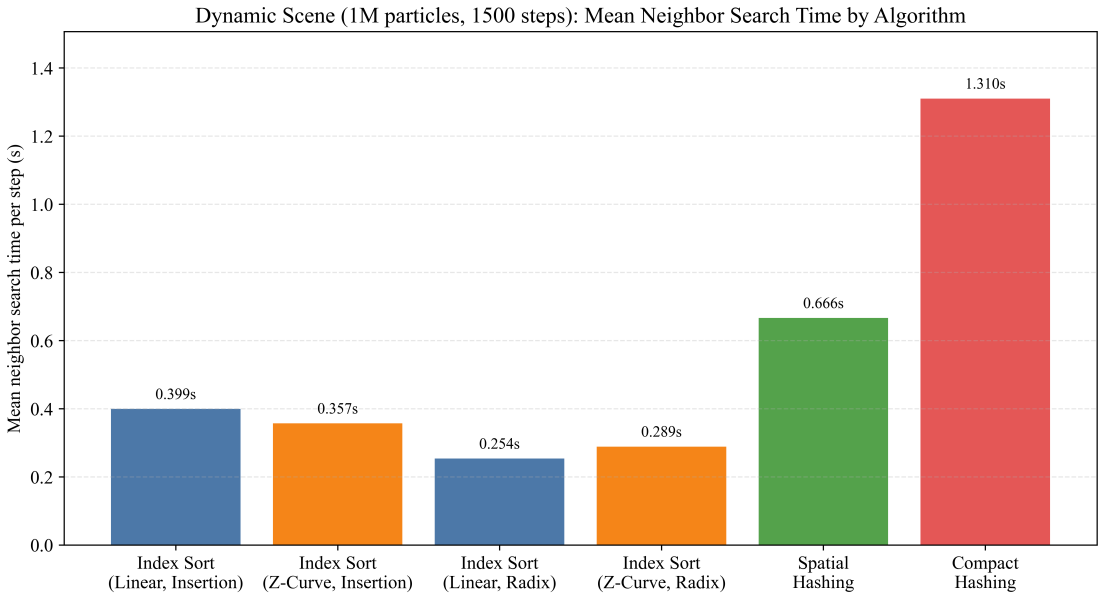


Figure 24: Mean neighbor search time per step (construct and query combined, first step excluded) for all six algorithms over the full 1500-step run. All four index sort variants are cheaper than either hashing method; Z-curve with insertion sort is cheaper on average than linear with insertion sort despite its larger individual spikes, since its advantage over the remainder of the run outweighs its rougher start.

5.2 Neighbor Representation Analysis

5.2.1 Compression

Figure 25 shows the measured effect: across a sweep from 100 thousand to 10 million particles (Z-curve indexing, radix sort), the compressed lists are consistently

about 57.5% smaller than the raw, 4-bytes-per-neighbor representation, essentially independent of scene size – expected, since the achievable compression depends on the local distribution of rank gaps between same-cell neighbors, not on the total particle count. This is lower than the 87% [BGT20] report for their own (bit-packed) encoding, consistent with the simplification described in Section 4.

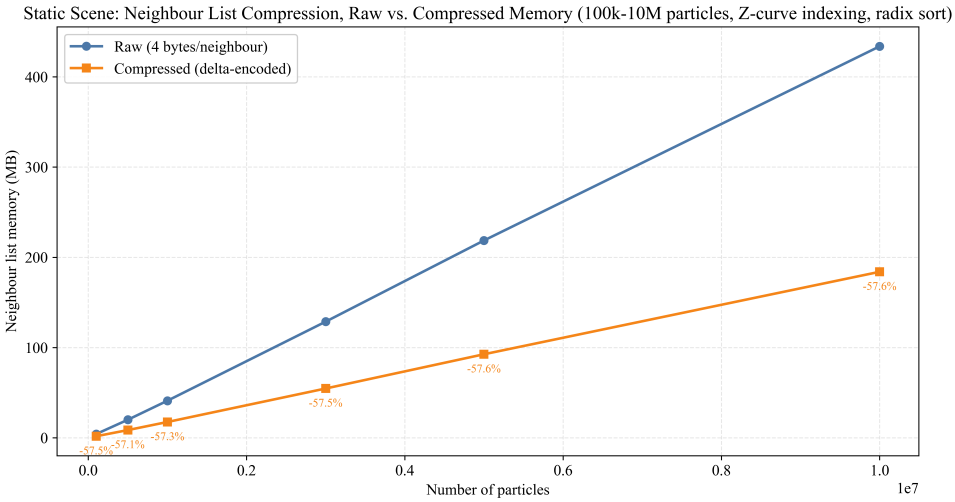


Figure 25: Raw vs. compressed neighbor list memory for a static scene, 100 thousand to 10 million particles (Z-curve indexing, radix sort). The achieved reduction (annotated per point) stays close to 57.5% across the whole range.

This reduction applies only to the neighbor lists themselves, not to the simulation's total memory. Comparing against the memory-scaling measurements above (Figure 18) shows that the raw neighbor lists account for roughly 13% of total memory at 1 million particles, growing to about 20% by 5-10 million – the rest being particle attributes (position, velocity, density, pressure), boundary particle data, and the index sort's own handles and cell references. A 57.5% reduction applied to that 20% share works out to roughly a 12% reduction in the simulation's total memory footprint at scale: a real and measurable win, but not one that removes or substantially moves the physical-memory ceiling shown in Figure 18 on its own. The remaining gap to [BGT20]'s reported effect is attributable both to the simplified, non-bit-packed encoding used here and to their additional cell-linked-list variant (Section 4), which targets the cell-reference memory rather than the neighbor lists and was not implemented in this work.

6 Conclusions

This thesis implemented a 2D SPH fluid solver together with six neighbor search variants: index sort with linear or Z-curve indexing, each paired with insertion or radix sort, and also spatial and compact hashing (Section 3). All six share the same solver, so they can be compared directly. They were measured both on

freshly constructed static scenes and inside a running dynamic simulation (Section 5).

On static scenes, index sort with radix sort clearly outperformed both hashing methods and scaled almost linearly with the number of particles. Z-curve indexing was slightly faster than linear indexing here, because of its better query-time cache locality (Figure 14). Insertion sort told a different story. It was cheap when its input was already close to sorted, which is what linear indexing's row-by-row generation order happens to produce. Z-curve indexing has no such structure, so its input is close to random, which pushes insertion sort toward its actual $O(n^2)$ worst case instead. Either way, insertion sort was impractical at scale: 430.6 s vs. 4.0 s for the same 10 million particles with radix sort. Between the two hashing methods, spatial hashing was consistently faster, but it also used roughly 2.7 times more memory than compact hashing – so compact hashing's memory saving comes at a real cost in speed, not for free. Pushing index sort with radix sort to 80 million particles showed that the real limit is memory, not compute: real memory usage crossed the test machine's 8 GB of RAM at around 36 million particles and reached 17.3 GB by 80 million, kept running only because the operating system fell back on memory compression and swap.

The dynamic setting changed this ranking in an interesting way. All four index sort variants stayed cheaper than either hashing method, but Z-curve with insertion sort – by far the most expensive variant in the static, cold-build setting – became cheaper on average than linear with insertion sort. This is because most steps only need to fix up the handful of particles that actually crossed a cell boundary, rather than rebuild everything from scratch. This also showed where each family of algorithms actually spends its time. Index sort is query-dominated once it is warm, at 63-87% of its per-step cost, since its handles are already sorted from the previous step. Spatial and compact hashing are the opposite: construct-dominated, at 56% and 71% respectively, since neither has an incremental update path and both rebuild their entire table from scratch every step. This is also why the hashing methods stayed noisy throughout the whole run instead of settling down like index sort did – there is no warm state for them to settle into.

Reviewing a neighbor list compression scheme from related work (Section 4) showed that index sort's cell-sorted handle ordering makes it a natural fit for delta-encoding. Spatial and compact hashing do not have an equivalent global ordering to exploit, so they cannot use this scheme the same way. A simplified, non-bit-packed implementation of the scheme achieved a consistent 57.5% reduction in neighbor list memory. This is lower than the 87% reported for the original bit-packed encoding, but it confirms that the same underlying idea works in this codebase too. Neighbor lists are only one part of a simulation's total memory footprint, so this translates to roughly a 12% reduction overall at scale – a real saving, but one that does not move the physical memory ceiling observed above on its own.

Taken together, these results suggest index sort with radix sort as the most broadly effective choice among the algorithms compared in this implementation, Z-curve indexing in particular given its advantage in the dynamic setting. Spatial hashing is only preferable when its higher memory cost is acceptable, and com-

pact hashing mainly when its simplicity is needed for its own sake. Beyond the scope of this thesis, three directions stand out. First, closing the gap to the 87% compression figure from related work by packing several control codes into a single byte, as the original scheme does, instead of storing one control code per byte. Second, implementing that work's cell-linked-list variant, which is reported at roughly 9× less memory than compact hashing (Section 4.1) but was only reviewed, not implemented, in this thesis. Third, giving spatial and compact hashing an incremental update path of their own instead of rebuilding from scratch every step, to see whether the construct-time cost identified here as their main dynamic-scene weakness can be reduced the same way index sort already avoids it.

A Boundary Handling

Boundary particles are static (Section 3). They never move, and there is no fluid on the other side of the wall to interpolate from, so they never get a physically meaningful density or pressure of their own. But they still act as neighbors of nearby fluid particles in the density and pressure computations, so their contribution has to be approximated somehow. This section describes the specific approximation used here, following the general particle-based, mass/volume-corrected boundary treatment for SPH [AIA+12].

A.1 Mirroring

Boundary particles never get a pressure value from the state equation (13), since they have no meaningful density deviation of their own. Instead, in the boundary term of Equation (12), a fluid particle i's own pressure p_i is *mirrored* onto the boundary neighbor b in place of the undefined p_b. This guarantees the pressure acceleration always pushes the fluid particle away from the wall instead of through it, without boundary particles ever needing to solve their own state equation.

Only the pressure term is mirrored, though. ρ_b in Equation (12) is not derived from ρ_i, and it is not computed for the boundary particle from its own neighborhood via Equation (11) either – boundary particles never go through the density computation at all. So ρ_b just stays fixed at ρ_0, the value it was constructed with.

A.2 Correction Factors

γ_1 in Equation (11) compensates for the fact that a boundary particle's own mass alone usually under- or overestimates the "missing" fluid mass its wall represents. γ_2 in Equation (12) plays the same role for the boundary contribution to the pressure acceleration. Both are fixed, empirically chosen constants applied the same way to every boundary contribution ($\gamma_1 = 1.2$, $\gamma_2 = 1.1$), not derived per particle.

B Simulation Parameters

The three main simulation parameters – gas stiffness k, time step Δt, and viscosity ν – are tuned per scene, not derived analytically. The criteria below come from earlier tuning experiments for this solver [Nah26b]:

- **Gas stiffness k.** Chosen high enough that the density error – how far the average fluid density drifts from the rest density ρ_0 – settles to around 0.1%-0.3% once the fluid is at rest, but not so high that it causes numerical instability.
- **Time step Δt.** Chosen so the Courant-Friedrichs-Lewy (CFL) condition holds: no particle moves farther than roughly its own size in a single step.
- **Viscosity ν.** Chosen per scene to match the intended fluid behavior – damping relative particle velocities enough to look right, without adding unwanted numerical oscillation.

C Implementation

C.1 Parallelization

Most of the per-particle work in this implementation runs in parallel across CPU cores using OpenMP (`#pragma omp parallel for`). This section covers where that works, why it works there, and where it does not.

C.1.1 What Parallelizes, and Why

The main simulation step – clearing forces, computing density and pressure, computing the pressure, viscosity and surface tension accelerations, and integrating position and velocity – is parallelized over particles. Each particle only reads shared data (its neighbor list, and its neighbors' density, pressure and position) and only writes to its own fields. Since no two threads ever write to the same particle, this parallelizes without needing any extra synchronization. The same holds for the visualization step, since each particle's color and vertex data are computed independently, and for the query step of every neighbor search structure used in this work (Section 3): index sort's `findNeighbours`, and spatial and compact hashing's neighbor queries, all only write to the particle currently being queried.

Constructing the hash tables is also parallelized over particles, but this needs more care. A hash table entry can be shared by several particles, so two particles hashed on two different threads can end up inserting into the same entry at the same time. Without synchronization that is a data race. Here, it actually showed up as a rare crash that only appeared once the particle count was high enough for a collision to happen within one frame. The fix: keep computing each particle's cell index and hash key in parallel, since that part is cheap and independent, but only do the actual entry insertion inside an OpenMP critical section.

C.1.2 What Does Not Parallelize

Sorting the handles in index sort (Section 3) is not parallelized, for either sorting method, but for two different reasons. Insertion sort is sequential by nature: inserting one handle means shifting it past however many handles are already sorted, so each step depends on the exact result of the previous one. Splitting that across threads would need a different, more complex parallel sorting algorithm altogether. Radix sort is single-threaded here too, but for a different reason: both of its counting passes build a prefix sum over 16-bit buckets (`count[i] += count[i - 1]`), and the placement pass afterward depends on that exact order to stay stable. Radix sort can be parallelized in general, for example by computing partial counts per thread and merging them before the prefix sum, but this implementation does not do that.

So when Section 5 compares insertion sort and radix sort for constructing the sorted handle list, the number of CPU cores does not matter there. Both run on a single core, so any gap between them comes from algorithmic complexity, not parallelization.

C.2 Hardware

The SPH fluid solver was developed and tested on a MacBook Air M3 with 8 GB of RAM and an Apple M3 chip. The M3 chip is a System on a Chip (SoC) that integrates the CPU, GPU, unified memory, and other components into a single chip. The unified memory architecture, shared between the CPU, GPU and accelerators, aims to increase bandwidth, improve efficiency, reduce memory latency and simplify memory management [HHPM25, Nah26b].

D Sketch-to-Scene Import Pipeline

This appendix gives a brief overview of the sketch-to-scene import pipeline used to generate several of the scenes in this work (Section 5): a way to turn hand-drawn artwork into particle scenes with arbitrary boundary shapes, instead of hand-coding geometry in C++.

D.1 Pipeline

A scene is sketched as a flat-color image (e.g. exported from Adobe Fresco): solid blue-family fills mark fluid, dark strokes mark boundary. The conversion script (`tools/scene_import/sketch_to_scene.py`) turns this into a particle scene file through the steps in Figure 26: threshold the image into fluid/boundary masks, keep a grid of candidate fluid points that fall inside the fluid mask, trace and resample the boundary strokes into evenly-spaced particles, then filter out any fluid sitting too close to a wall or in a corridor narrower than the kernel support – turning an excluded gap into solid wall rather than leaving a hole. The output is a plain-text file, one particle per line (`x y type`), read directly by `InitializeFromSceneFile` in `ParticleSettings.cpp`, so no scene-specific C++

is needed.

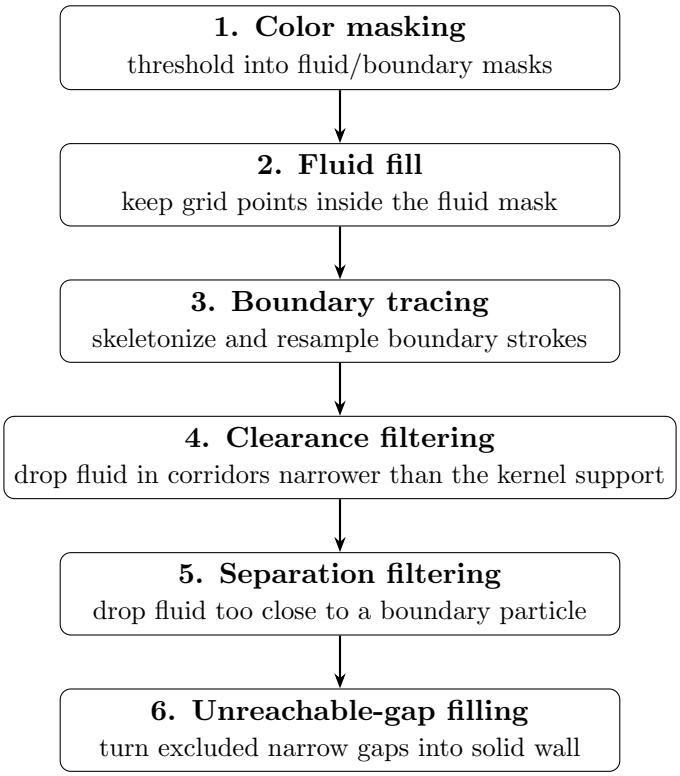


Figure 26: The sketch-to-scene pipeline. Each sketch passes through these steps, in order, to produce a particle scene file.

D.2 Example Scenes

Table 1 lists the sketch-imported scenes used in this work, with their approximate fluid and boundary particle counts.

Scene	Fluid particles	Boundary particles
Downwards (winding channel)	98,006	8,380
Underground (braided channel)	82,275	38,695
Tsunami (coastal wave)	48,664	4,766

Table 1: Sketch-imported scenes and their particle counts.

E Resources

Claude (Anthropic) was used as an assistant during the writing of this thesis in the following aspects: for visualizing scene geometry and pipeline steps (Appendix D), for generating the performance plots in Section 5, and for LaTeX formatting and general feedback.

References

[AIA+12] N. Akinci, M. Ihmsen, G. Akinci, B. Solenthaler, and M. Teschner. Versatile rigid-fluid coupling for incompressible SPH. *ACM Trans. Graph.*, 31(4):62:1–62:8, 2012.

[BGT20] S. Band, C. Gissler, and M. Teschner. Compressed neighbour lists for sph. *Computer Graphics Forum*, 39(1):531–542, 2020.

[Bro85] L. Brookshaw. A method of calculating radiative heat diffusion in particle simulations. *Publications of the Astronomical Society of Australia*, 6(2):207–210, 1985.

[BT07] M. Becker and M. Teschner. Weakly compressible SPH for free surface flows. In Michael Gleicher and Daniel Thalmann, editors, *Proceedings of the 2007 ACM SIGGRAPH/Eurographics Symposium on Computer Animation, SCA 2007, San Diego, California, USA, August 2-4, 2007*, pages 209–217. Eurographics Association, 2007.

[FFWL+22] J. Fernández-Fernández, L. Westhofen, F. Löschner, S. Jeske, A. Longva, and J. Bender. Fast octree neighborhood search for sph simulations. *ACM Trans. Graph.*, 41(6), November 2022.

[GM77] R. Gingold and J. Monaghan. Smoothed particle hydrodynamics: theory and application to non-spherical stars. *Monthly Notices of the Royal Astronomical Society*, 181(3):375–389, December 1977.

[GM82] R. Gingold and J. Monaghan. Kernel estimates as a basis for general particle methods in hydrodynamics. *Journal of Computational Physics*, 46(3):429–453, 1982.

[GPB+19] C. Gissler, A. Peer, S. Band, J. Bender, and M. Teschner. Interlinked sph pressure solvers for strong fluid-rigid coupling. *ACM Trans. Graph.*, 38(1), January 2019.

[Gre10] S. Green. Particle simulation using cuda. Nvidia whitepaper, NVIDIA, 2010.

[GSSP10] P. Goswami, P. Schlegel, B. Solenthaler, and R. Pajarola. Interactive sph simulation and rendering on the gpu. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation*, pages 55–64. Eurographics Association, 2010.

[HHPM25] P. Hübner, A. Hu, I. Peng, and S. Markidis. Apple vs. Oranges: Evaluating the Apple Silicon M-Series SoCs for HPC Performance and Efficiency. *arXiv e-prints*, page arXiv:2502.05317, February 2025.

[IABT11] M. Ihmsen, N. Akinci, M. Becker, and M. Teschner. A parallel SPH implementation on multi-core cpus. *Comput. Graph. Forum*, 30(1):99–112, 2011.

[ICS+14] M. Ihmsen, J. Cornelis, B. Solenthaler, C. Horvath, and M. Teschner. Implicit incompressible SPH. *IEEE Trans. Vis. Comput. Graph.*, 20(3):426–435, 2014.

[KBST19] D. Koschier, J. Bender, B. Solenthaler, and M. Teschner. Smoothed particle hydrodynamics techniques for the physics based simulation of fluids and solids. In Wenzel Jakob and Enrico Puppo, editors, *40th Annual Conference of the European Association for Computer Graphics, Eurographics 2019 - Tutorials, Genoa, Italy, May 6-10, 2019*, pages 1–41. Eurographics Association, 2019.

[KBST22] D. Koschier, J. Bender, B. Solenthaler, and M. Teschner. A survey on SPH methods in computer graphics. *Comput. Graph. Forum*, 41(2):737–760, 2022.

[Knu98] D. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, 2nd edition, 1998.

[LL10] M. Liu and G. Liu. Smoothed particle hydrodynamics (sph): an overview and recent developments. *Archives of Computational Methods in Engineering*, 17(1):25–76, 2010.

[LSY01] R. Lee, Z. Shi, and X. Yang. Efficient permutation instructions for fast software cryptography. *IEEE Micro*, 21(6):56–69, 2001.

[Luc77] L. Lucy. A numerical approach to the testing of the fission hypothesis. *The Astronomical Journal*, 82:1013–1024, December 1977.

[Mor66] G. Morton. A computer oriented geodetic data base and a new technique in file sequencing. Technical report, International Business Machines Company, New York, 1966.

[MST04] M. Müller, S. Schirm, and M. Teschner. Interactive blood simulation for virtual surgery based on smoothed particle hydrodynamics. *Technology and health care : official journal of the European Society for Engineering and Medicine*, 12:25–31, 02 2004.

[Nah26a] A. Nahhas. Smoothed Particle Hydrodynamics. Seminar report, Chair for Computer Graphics, Albert-Ludwigs-Universität Freiburg, July 2026.

[Nah26b] A. Nahhas. SPH fluid solver in (2d). Bachelor project report, Chair for Computer Graphics, Albert-Ludwigs-Universität Freiburg, February 2026.

[OD08] J. Onderik and R. Durikovic. Efficient neighbor search for particle-based fluids. *Journal of Applied Mathematics, Statistics and Informatics*, 4(1):29–43, 2008.

[PF01] V. Pascucci and R. Frank. Global static indexing for real-time exploration of very large regular grids. In *Proceedings of the 2001 ACM/IEEE Conference on Supercomputing*, SC '01, Denver, CO, USA, 2001. ACM.

[PGBT18] A. Peer, C. Gissler, S. Band, and M. Teschner. An implicit sph formulation for incompressible linearly elastic solids. *Computer Graphics Forum*, 37(6):135–148, 2018.

[Pri12] D. Price. Smoothed particle hydrodynamics and magnetohydrodynamics. *Journal of Computational Physics*, 231(3):759–794, February 2012.

[RYL+18] B. Ren, X. Yang, M. Lin, N. Thuerey, M. Teschner, and C. Li. Visual simulation of multiple fluids in computer graphics: A state-of-the-art report. *J. Comput. Sci. Technol.*, 33(3):431–451, 2018.

[Sew54] H. Seward. Information sorting in the application of electronic digital computers to business operations. Report R-232, MIT Digital Computer Laboratory, 1954.

[THM+03] M. Teschner, B. Heidelberger, M. Müller, D. Pomerantes, and M. Gross. Optimized spatial hashing for collision detection of deformable objects. In Thomas Ertl, editor, *8th International Fall Workshop on Vision, Modeling, and Visualization, VMV 2003, München, Germany, November 19-21, 2003*, pages 47–54. Aka GmbH, 2003.